

```

1  package com.example.demo.model.bo.insp;
2
3  import java.time.LocalDateTime;
4
5  import jakarta.persistence.Column;
6  import jakarta.persistence.EmbeddedId;
7  import jakarta.persistence.Entity;
8  import jakarta.persistence.Table;
9  import lombok.AllArgsConstructor;
10 import lombok.Builder;
11 import lombok.Data;
12 import lombok.EqualsAndHashCode;
13 import lombok.NoArgsConstructor;
14
15 @NoArgsConstructor
16 @AllArgsConstructor
17 @Data
18 @EqualsAndHashCode
19 @Builder
20 @Entity
21 @Table(name = "PMS_BATCH_TRACK")
22 public class PMS_Batch_Track {
23
24     @EmbeddedId
25     private PMS_Batch_Track_PK btrack_pk;
26
27     @Column(name = "PAINT_TYPE", length = 200)
28     private String paintType;
29
30     @Column(name = "PAINT_TC_DOC", length = 200)
31     private String paint_tc_doc;
32
33     @Column(name = "CALL_STATUS", length = 10)
34     private String callStatus;
35
36     // Calculated by System when TPI Clear the Inspection !!!
37     @Column(name = "CALL_CLR_DT_TM")
38     private LocalDateTime callClrDtTm;
39
40     // Set by Oracle Procedure
41     @Column(name = "IS_ALLOW_NEXT_CALL_TYPE", length = 100)
42     private String isProceedNext;
43
44     // Calculated by System when TPI Clear the Inspection !!!
45     @Column(name = "NEXT_CALL_TIME")
46     private LocalDateTime nextCallTime;
47
48     @Column(name = "ACT_BY", length = 10)
49     private String actBy;
50 }
51
52 package com.example.demo.dao.insp;
53
54 import java.time.LocalDateTime;
55 import java.util.List;
56
57 import org.springframework.data.jpa.repository.JpaRepository;
58 import org.springframework.data.jpa.repository.Modifying;
59 import org.springframework.data.jpa.repository.Query;
60 import org.springframework.data.repository.query.Param;
61 import org.springframework.stereotype.Repository;
62
63 import com.example.demo.model.bo.insp.PMS_Batch_Track;
64 import com.example.demo.model.bo.insp.PMS_Batch_Track_PK;

```

```

65 import com.example.demo.model.dto.insp.PMS_Batch_Track_Dto;
66
67 import jakarta.transaction.Transactional;
68
69 @Repository
70 public interface PMS_Batch_Track_Dao extends JpaRepository<PMS_Batch_Track,
PMS_Batch_Track_PK> {
71
72     /**
73      * Mark Call Completion for A batch + Call_type + vendor
74      *
75      */
76     @Transactional
77     @Modifying
78     @Query("""
79         UPDATE
80             PMS_Batch_Track p
81         SET
82             p.callClrDtTm = :currentTime,
83             p.callStatus = :callStatus,
84             p.actBy = :actBy
85         WHERE
86             p.btrack_pk.callRefNum = :callRefNum AND
87             p.btrack_pk.vcode = :vcode AND
88             p.btrack_pk.inspFyYear = :inspFyYear AND
89             p.btrack_pk.callType = :callType
90             AND (p.callStatus IS NULL OR p.callStatus <> 'REWORK')
91         """)
92     int update_call_completion_for_a_call_type(@Param("currentTime") LocalDateTime
currentTime,
93         @Param("callRefNum") Integer callRefNum, @Param("vcode") String vcode,
94         @Param("inspFyYear") String inspFyYear, @Param("callType") String callType,
95         @Param("callStatus") String call_status, @Param("actBy") String actBy);
96
97     /**
98      * Update Next Call Time during ADDI_INFO Stage
99      */
100    @Transactional
101    @Modifying
102    @Query("""
103        UPDATE
104            PMS_Batch_Track p
105        SET
106            p.nextCallTime = :nextCallTime,
107            p.btrack_pk.callReqDtTm = :currentTime
108        WHERE
109            p.btrack_pk.callRefNum = :callRefNum AND
110            p.btrack_pk.vcode = :vcode AND
111            p.btrack_pk.inspFyYear = :inspFyYear AND
112            p.btrack_pk.callType = :callType
113            AND (p.callStatus IS NULL OR p.callStatus <> 'REWORK')
114        """)
115    int update_next_call_time(@Param("currentTime") LocalDateTime currentTime, @Param(
"nextCallTime") LocalDateTime nextCallTime,
116        @Param("callRefNum") Integer callRefNum, @Param("vcode") String vcode,
117        @Param("inspFyYear") String inspFyYear, @Param("callType") String callType);
118
119    /**
120     * Completed{ACC} /WIP {NULL} Call-Type for a given given batch (Call History
121     * Tracking!!!)
122     */
123    @Query("""
124        SELECT
125            new com.example.demo.model.dto.insp.PMS_Batch_Track_Dto(p)

```

```

126         FROM
127             PMS_Batch_Track p
128         WHERE
129             p.btrack_pk.vcode = :vcode
130             AND (p.callStatus = 'ACC' OR p.callStatus IS NULL)
131         """)
132     List<PMS_Batch_Track_Dto> get_call_type_history_for_a_vendor(@Param("vcode") String
vcode);
133
134     /**
135      * Get Completed call-list for printing & signing
136      */
137     @Query("""
138         SELECT
139             new com.example.demo.model.dto.insp.PMS_Batch_Track_Dto(p)
140         FROM
141             PMS_Batch_Track p
142         WHERE
143             p.callStatus = 'ACC'
144         ORDER BY
145             p.btrack_pk.callRefNum ASC
146         """)
147     List<PMS_Batch_Track_Dto> get_completed_call_list();
148
149     /**
150      * Test Update - Delete this Query-Method
151      */
152     @Transactional
153     @Modifying
154     @Query("""
155         UPDATE
156             PMS_Batch_Track p
157         SET
158             p.callStatus = :callStatus
159         """)
160     int update_test(@Param("callStatus") String call_status);
161
162 }
163
164 package com.example.demo.model.dto.insp;
165
166 import java.time.LocalDateTime;
167 import java.time.format.DateTimeFormatter;
168
169 import com.example.demo.model.bo.insp.PMS_Batch_Track;
170
171 import lombok.AllArgsConstructor;
172 import lombok.Builder;
173 import lombok.Data;
174 import lombok.NoArgsConstructor;
175
176 @NoArgsConstructor
177 @AllArgsConstructor
178 @Data
179 @Builder
180 public class PMS_Batch_Track_Dto {
181
182     // Fields from EmbeddedId
183     private String vcode;
184     private Integer callRefNum;
185     private String inspFyYear;
186     private String callType;
187
188     // Fields from main entity

```

```

189     private String paintType;
190     private String callClrDtTm;
191     private String isProceedNext;
192     private LocalDateTime nextCallTime;
193
194     // ✅ Custom constructor
195     public PMS_Batch_Track_Dto(PMS_Batch_Track entity) {
196
197         if (entity != null && entity.getBtrack_pk() != null) {
198             this.vcode = entity.getBtrack_pk().getVcode();
199             this.callRefNum = entity.getBtrack_pk().getCallRefNum();
200             this.inspFyYear = entity.getBtrack_pk().getInspFyYear();
201             this.callType = entity.getBtrack_pk().getCallType();
202
203         }
204
205         // Format: DD-MM-YY HH:mm:ss AM/PM
206         DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd-MM-yy hh:mm:ss a");
207
208         String temp_dt_tm = "";
209
210         if (entity.getCallClrDtTm() != null) {
211             temp_dt_tm = entity.getCallClrDtTm().format(formatter);
212         }
213
214         this.callClrDtTm = temp_dt_tm;
215
216         this.paintType = entity.getPaintType();
217         this.isProceedNext = entity.getIsProceedNext();
218         this.nextCallTime = entity.getNextCallTime();
219     }
220 }
221
222 package com.example.demo.model.bo.insp;
223
224 import java.io.Serializable;
225 import java.time.LocalDateTime;
226
227 import jakarta.persistence.Column;
228 import jakarta.persistence.Embeddable;
229 import lombok.AllArgsConstructor;
230 import lombok.Builder;
231 import lombok.Data;
232 import lombok.EqualsAndHashCode;
233 import lombok.NoArgsConstructor;
234
235 @NoArgsConstructor
236 @AllArgsConstructor
237 @Data
238 @EqualsAndHashCode
239 @Builder
240 @Embeddable
241 public class PMS_Batch_Track_PK implements Serializable {
242
243     @Column(name = "v_code", length = 10)
244     private String vcode;
245
246     @Column(name = "CALL_REF_NUL", precision = 10)
247     private Integer callRefNum;
248
249     @Column(name = "INSP_FY_YEAR", length = 4)
250     private String inspFyYear;
251
252     @Column(name = "CALL_REQ_DT_TM")

```

```

253     private LocalDateTime callReqDtTm;
254
255     @Column(name = "CALL_TYPE", length = 200)
256     private String callType;
257
258 }
259
260 package com.example.demo.model.bo.insp;
261
262 import java.math.BigDecimal;
263 import java.time.LocalDate;
264 import java.time.LocalDateTime;
265 import java.time.LocalTime;
266
267 import jakarta.persistence.Column;
268 import jakarta.persistence.EmbeddedId;
269 import jakarta.persistence.Entity;
270 import jakarta.persistence.Table;
271 import lombok.AllArgsConstructor;
272 import lombok.Builder;
273 import lombok.Data;
274 import lombok.NoArgsConstructor;
275
276 @NoArgsConstructor
277 @AllArgsConstructor
278 @Data
279 @Builder
280 @Entity
281 @Table(name = "PMS_BLAST_2_5")
282 public class Pms_Blast_2_5 {
283
284     /* ----- */
285
286     @EmbeddedId
287     private Pms_Blast_2_5_PK blast_2_5_pk;
288
289     /* ----- */
290
291     private String duDesc;
292     private Double duCallQty;
293
294     @Column(name = "DRY_BULB_T", precision = 6, scale = 3)
295     private BigDecimal dryBulbT;
296
297     @Column(name = "WET_BULB_T", precision = 6, scale = 3)
298     private BigDecimal wetBulbT;
299
300     @Column(name = "DEW_PT", precision = 6, scale = 3)
301     private BigDecimal dewPt;
302
303     private BigDecimal rh;
304
305     private LocalDate readDt;
306     private LocalTime readTimeSt;
307     private LocalTime readTimeEnd;
308
309     @Column(length = 250)
310     private String inspRmrk;
311
312     /* ----- */
313
314     @Column(name = "blast_medium", length = 25)
315     private String blast_medium;
316

```

```

317     @Column(name = "raw_matl_rust_grade", length = 1)
318     private String raw_matl_rust_grade;
319
320     @Column(name = "moist_cont", length = 25)
321     private String moist_cont;
322
323     @Column(name = "surf_cloride", length = 25)
324     private String surf_cloride;
325
326     @Column(name = "surf_prof_gauge_read")
327     private Double surf_prof_gauge_read;
328
329     @Column(name = "checked_dt_tm")
330     private LocalDateTime checked_dt_tm;
331
332     @Column(name = "dust_free_check", length = 1)
333     private String dust_free_check;
334
335     @Column(name = "visual_insp", length = 25)
336     private String visual_insp;
337
338     /* ----- */
339
340     @Column(length = 7)
341     private String inspClrBy;
342
343     private LocalDate inspClrDt;
344 }
345
346 package com.example.demo.dao.insp;
347
348 import java.util.List;
349
350 import org.springframework.data.jpa.repository.JpaRepository;
351 import org.springframework.data.jpa.repository.Query;
352 import org.springframework.stereotype.Repository;
353
354 import com.example.demo.model.bo.insp.Pms_Blast_2_5;
355 import com.example.demo.model.bo.insp.Pms_Blast_2_5_PK;
356 import com.example.demo.model.dto.insp.Pms_Tpi_Dynamic_Row_Dto;
357
358 @Repository
359 public interface Pms_Blast_2_5_Dao extends JpaRepository<Pms_Blast_2_5, Pms_Blast_2_5_PK>
360 {
361     /**
362      * Dynamic_Row_Data_For_A_Vendor_For_A_Call_Number_OR_Batch for
363      * 'Blasting'
364      */
365     @Query("""
366         SELECT
367             new com.example.demo.model.dto.insp.Pms_Tpi_Dynamic_Row_Dto(p, q)
368         FROM
369             Pms_Blast_2_5 p
370         LEFT JOIN
371             Po_Details q
372             ON p.blast_2_5_pk.poNum = q.po_det_pk.zzEbels
373             AND p.blast_2_5_pk.duNum = q.po_det_pk.zzEbels
374         WHERE
375             p.blast_2_5_pk.vcode = :vcode
376             AND p.blast_2_5_pk.callRefNum = :call_num
377         """)
378     public List<Pms_Tpi_Dynamic_Row_Dto> get_dynamic_row_data(String vcode, Integer
call_num);

```

```

379 }
380
381 package com.example.demo.model.bo.insp;
382
383 import java.io.Serializable;
384
385 import jakarta.persistence.Column;
386 import jakarta.persistence.Embeddable;
387 import lombok.AllArgsConstructor;
388 import lombok.Builder;
389 import lombok.Data;
390 import lombok.NoArgsConstructor;
391
392 @NoArgsConstructor
393 @AllArgsConstructor
394 @Data
395 @Builder
396 @Embeddable
397 public class Pms_Blast_2_5_PK implements Serializable {
398
399     @Column(name = "CALL_REF_NUM", precision = 10)
400     private Integer callRefNum;
401
402     @Column(name = "VCODE", length = 10)
403     private String vcode;
404
405     @Column(name = "PO_NUM", length = 10)
406     private String poNum;
407
408     @Column(name = "DU_NUM", precision = 5)
409     private Integer duNum;
410 }
411
412 package com.example.demo.model.vo.insp;
413
414 import java.time.LocalDateTime;
415 import java.util.List;
416
417 import com.fasterxml.jackson.annotation.JsonProperty;
418 import com.google.auto.value.AutoValue.Builder;
419
420 import lombok.AllArgsConstructor;
421 import lombok.Data;
422 import lombok.NoArgsConstructor;
423
424 @NoArgsConstructor
425 @AllArgsConstructor
426 @Data
427 @Builder
428 public class Pms_Blast_sa_2_5_Vo {
429
430     /*****/
431
432     @JsonProperty("frm_grp_ctrl_vcode")
433     private String frmGrpCtrlVcode;
434
435     @JsonProperty("frm_grp_ctrl_ref")
436     private Integer frmGrpCtrlRef;
437
438     @JsonProperty("frm_grp_ctrl_po")
439     private String frmGrpCtrlPo;
440
441     @JsonProperty("frm_grp_ctrl_call_type")
442     private String frmGrpCtrlCallType;

```

```

443
444     @JsonProperty("frm_grp_ctrl_paint_type")
445     private String frm_grp_ctrl_paint_type;
446
447     /*****/
448
449     @JsonProperty("blast_medium")
450     private String blast_medium;
451
452     @JsonProperty("raw_matl_rust_grade")
453     private String raw_matl_rust_grade;
454
455     @JsonProperty("moist_cont")
456     private String moist_cont;
457
458     @JsonProperty("surf_cloride")
459     private String surf_cloride;
460
461     @JsonProperty("surf_prof_gauge_read")
462     private Double surf_prof_gauge_read;
463
464     @JsonProperty("checked_dt_tm")
465     private LocalDateTime checked_dt_tm;
466
467     @JsonProperty("dust_free_check")
468     private String dust_free_check;
469
470     @JsonProperty("visual_insp")
471     private String visual_insp;
472
473     /*****/
474
475     @JsonProperty("row_form_group")
476     private List<Pms_Pow_tool_Vo_Row_Group> rowFormGroup;
477
478     /*****/
479 }
480
481 package com.example.demo.model.dto.insp;
482
483 import java.time.format.DateTimeFormatter;
484
485 import com.example.demo.model.bo.insp.PMS_Batch_Track;
486 import com.example.demo.model.bo.insp.Pms_Geo_Img;
487 import com.example.demo.model.bo.insp.Pms_Root;
488
489 import lombok.AllArgsConstructor;
490 import lombok.Builder;
491 import lombok.Data;
492 import lombok.NoArgsConstructor;
493
494 @NoArgsConstructor
495 @AllArgsConstructor
496 @Data
497 @Builder
498 public class Pms_Call_Report_Dto {
499
500     private String vcode;
501     private Integer callRefNum;
502     private String poNum;
503     private String wbs;
504     private Integer duNum;
505     private String du_code;
506     private String duDesc;

```



```

507 private Integer duCallQty;
508 private String paintScheme;
509 private String callType;
510 private String paintType;
511 private String callStatus;
512 private String callCreatedDtTm;
513 private String callClrDtTm;
514 private String nextCallTime;
515 private String actBy;
516 private String lati;
517 private String longi;
518 private String img_name;
519
520 /**
521  * Custom Constructor
522  */
523 public Pms_Call_Report_Dto(Pms_Root pmsRoot, PMS_Batch_Track batchTrack) {
524
525     DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd-MM-yy hh:mm a");
526
527     vcode = pmsRoot.getPms_root_pk().getVcode();
528     callRefNum = pmsRoot.getPms_root_pk().getCallRefNum();
529     poNum = pmsRoot.getPms_root_pk().getPoNum();
530     wbs = pmsRoot.getWbs();
531     duNum = pmsRoot.getPms_root_pk().getDuNum();
532     du_code = pmsRoot.getDu_code();
533     duDesc = pmsRoot.getDu_desc();
534     duCallQty = pmsRoot.getDuCallQTY();
535     paintScheme = pmsRoot.getPaintScheme();
536
537     if (batchTrack != null) {
538
539         callType = batchTrack.getBtrack_pk().getCallType();
540         paintType = batchTrack.getPaintType();
541
542         if (batchTrack.getCallStatus() != null) {
543             callStatus = batchTrack.getCallStatus();
544         } else {
545             callStatus = pmsRoot.getCallStatus();
546         }
547
548         if (batchTrack.getBtrack_pk().getCallReqDtTm() != null) {
549             callCreatedDtTm = batchTrack.getBtrack_pk().getCallReqDtTm().format(
                    formatter);
550         }
551
552         if (batchTrack.getCallClrDtTm() != null) {
553             callClrDtTm = batchTrack.getCallClrDtTm().format(formatter);
554         }
555
556         if (batchTrack.getNextCallTime() != null) {
557             nextCallTime = batchTrack.getNextCallTime().format(formatter);
558         }
559
560         actBy = batchTrack.getActBy();
561
562     } else {
563
564         callStatus = pmsRoot.getCallStatus();
565     }
566
567 }
568
569 /**

```

```

570     * Custom Constructor
571     */
572     public Pms_Call_Report_Dto(Pms_Root pmsRoot, PMS_Batch_Track batchTrack, Pms_Geo_Img
pms_geo_loc) {
573
574         DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd-MM-yy hh:mm a");
575
576         vcode = pmsRoot.getPms_root_pk().getVcode();
577         callRefNum = pmsRoot.getPms_root_pk().getCallRefNum();
578         poNum = pmsRoot.getPms_root_pk().getPoNum();
579         wbs = pmsRoot.getWbs();
580         duNum = pmsRoot.getPms_root_pk().getDuNum();
581         du_code = pmsRoot.getDu_code();
582         duDesc = pmsRoot.getDu_desc();
583         duCallQty = pmsRoot.getDuCallQTY();
584         paintScheme = pmsRoot.getPaintScheme();
585
586         if (vcode.equals("30001") && callRefNum == 1000000010) {
587             Integer x = 0;
588             x = x + 2;
589             System.out.println(x);
590         }
591
592         if (batchTrack != null) {
593
594             callType = batchTrack.getBtrack_pk().getCallType();
595             paintType = batchTrack.getPaintType();
596
597             if (batchTrack.getCallStatus() != null) {
598                 callStatus = batchTrack.getCallStatus();
599             } else {
600                 callStatus = pmsRoot.getCallStatus();
601             }
602
603             if (batchTrack.getBtrack_pk().getCallReqDtTm() != null) {
604                 callCreatedDtTm = batchTrack.getBtrack_pk().getCallReqDtTm().format(
formatter);
605             }
606
607             if (batchTrack.getCallClrDtTm() != null) {
608                 callClrDtTm = batchTrack.getCallClrDtTm().format(formatter);
609             }
610
611             if (batchTrack.getNextCallTime() != null) {
612                 nextCallTime = batchTrack.getNextCallTime().format(formatter);
613             }
614
615             actBy = batchTrack.getActBy();
616
617         } else {
618
619             callStatus = pmsRoot.getCallStatus();
620         }
621
622         if (pms_geo_loc != null) {
623             this.lati = pms_geo_loc.getGeo_lat();
624             this.longi = pms_geo_loc.getGeo_long();
625             this.img_name = pms_geo_loc.getImg_name();
626         }
627
628     }
629
630 }
631

```

```

632 package com.example.demo.model.vo.insp;
633
634 import java.util.List;
635
636 import org.springframework.web.multipart.MultipartFile;
637
638 import lombok.AllArgsConstructor;
639 import lombok.Builder;
640 import lombok.Data;
641 import lombok.NoArgsConstructor;
642
643 @NoArgsConstructor
644 @AllArgsConstructor
645 @Data
646 @Builder
647 public class Pms_Call_Vo {
648
649     private String callType;
650     private String paintType;
651     private String poNo;
652     private String paintSchmRef;
653     private String project;
654     private String woNo;
655     private String callDtTm;
656     private MultipartFile docFile;
657     private List<Pms_Call_Vo_row_form_Array> duCall;
658 }
659
660 package com.example.demo.model.vo.insp;
661
662 import lombok.AllArgsConstructor;
663 import lombok.Builder;
664 import lombok.Data;
665 import lombok.NoArgsConstructor;
666
667 @NoArgsConstructor
668 @AllArgsConstructor
669 @Data
670 @Builder
671 public class Pms_Call_Vo_row_form_Array {
672
673     private String du;
674     private String du_code;
675     private String desc;
676     private int qty;
677     private String du_wbs;
678 }
679
680 package com.example.demo.controller.depts.insp;
681
682 import java.util.ArrayList;
683 import java.util.Comparator;
684 import java.util.List;
685 import java.util.stream.Collectors;
686
687 import org.springframework.http.ResponseEntity;
688 import org.springframework.security.access.annotation.Secured;
689 import org.springframework.security.core.Authentication;
690 import org.springframework.web.bind.annotation.GetMapping;
691 import org.springframework.web.bind.annotation.PathVariable;
692 import org.springframework.web.bind.annotation.PostMapping;
693 import org.springframework.web.bind.annotation.RequestBody;
694 import org.springframework.web.bind.annotation.RequestMapping;
695 import org.springframework.web.bind.annotation.RequestParam;

```

```

696 import org.springframework.web.bind.annotation.RestController;
697 import org.springframework.web.multipart.MultipartFile;
698
699 import com.example.demo.model.dto.Address_Prime_Data_Dto;
700 import com.example.demo.model.dto.insp.PMS_Batch_Track_Dto;
701 import com.example.demo.model.dto.insp.Pms_Call_Report_Dto;
702 import com.example.demo.model.dto.insp.Pms_Du_Img_GLoc_Dto;
703 import com.example.demo.model.dto.insp.Pms_Fnl_Qty_Dto;
704 import com.example.demo.model.dto.insp.Pms_Po_Details_Dto;
705 import com.example.demo.model.dto.insp.Pms_Root_Dto;
706 import com.example.demo.model.dto.insp.Pms_Root_Po_Du_Sum_Qty_Dto;
707 import com.example.demo.model.dto.insp.Pms_Tpi_Dynamic_Row_Dto;
708 import com.example.demo.model.vo.insp.Pms_Blast_sa_2_5_Vo;
709 import com.example.demo.model.vo.insp.Pms_Pow_tool_Vo;
710 import com.example.demo.model.vo.insp.Pms_Prime_I_II_Vo;
711 import com.example.demo.service.Address_Service;
712 import com.example.demo.service.insp.PMS_Serv;
713
714 @RestController
715 @RequestMapping("/insp/pms")
716 public class Pms_Controller {
717
718     private final PMS_Serv pms_serv;
719     private final Address_Service addr_serv;
720
721     public Pms_Controller(PMS_Serv pms_serv, Address_Service addr_serv) {
722         this.pms_serv = pms_serv;
723         this.addr_serv = addr_serv;
724     }
725
726     /**
727      * Returns PO Details as first & sb-vendor
728      *
729      * @param vcode
730      * @return
731      */
732     @GetMapping("/get_po_pms")
733     @Secured({ "ROLE_OUTS_A", "ROLE_OUTS_S" })
734     public ResponseEntity<List<Pms_Po_Details_Dto>> get_pos_pms_vendor(Authentication
authened_obj) {
735
736         String vendor_code = "00000" + authened_obj.getName();
737
738         // Fetch the result from the service method
739         List<Pms_Po_Details_Dto> result_1_st_vendor_po = pms_serv.get_all_po_by_vendor(
vendor_code);
740         List<Pms_Po_Details_Dto> result_2_st_vendor_po = pms_serv.get_po_for_2_nd_vendor(
vendor_code);
741
742         List<Pms_Po_Details_Dto> combinedList = new ArrayList<>();
743
744         combinedList.addAll(result_1_st_vendor_po);
745         combinedList.addAll(result_2_st_vendor_po);
746
747         List<Pms_Po_Details_Dto> result = combinedList.stream().sorted(
Comparator.comparing(Pms_Po_Details_Dto::getZzEbeln).thenComparing(
Pms_Po_Details_Dto::getZzEbelp))
.collect(Collectors.toList());
749
750         // If result is null or empty, you might want to handle that as well
751         if (result == null || result.isEmpty()) {
752             return ResponseEntity.noContent().build(); // 204 No Content
753         }
754     }
755

```

```

756         // Return the result wrapped in a ResponseEntity with HTTP status OK (200)
757         return ResponseEntity.ok(result);
758     }
759
760     /**
761     * Get all committed-Calls for approval/rejection
762     */
763     @GetMapping("/all_fresh_calls")
764     @Secured({ "ROLE_OUTS_A", "ROLE_OUTS_S" })
765     public ResponseEntity<List<Pms_Root_Dto>> get_all_fresh_calls(Authentication
authed_obj) {
766
767         var result = pms_serv.get_all_fresh_calls(authed_obj.getName());
768
769         return ResponseEntity.ok(result);
770     }
771
772     /**
773     * Get all BATCHED-Calls for Feeding Additional Data for the Batch by Vendor
774     */
775     @GetMapping("/all_batched_calls")
776     @Secured({ "ROLE_OUTS_A", "ROLE_OUTS_S" })
777     public ResponseEntity<List<Pms_Root_Dto>> get_all_batched_calls(Authentication
authed_obj) {
778
779         var result = pms_serv.get_all_batched_calls(authed_obj.getName());
780
781         return ResponseEntity.ok(result);
782     }
783
784     /**
785     * Get all Additional-Info-Provided-Calls for approval/rejection by TPI
786     */
787     @GetMapping("/all_addi_calls")
788     @Secured({ "ROLE_OUTS_A", "ROLE_OUTS_S", "ROLE_INSA_A", "ROLE_INSA_D" })
789     public ResponseEntity<List<Pms_Root_Dto>> get_all_committed_calls() {
790
791         var result = pms_serv.get_all_addi_calls();
792
793         return ResponseEntity.ok(result);
794     }
795
796     /*****
797
798     /**
799     * Get Row-Data for 'Power-Tool-Cleaning'
800     */
801     @GetMapping("/rd_ptc/{vcode}/{batch}")
802     public ResponseEntity<List<Pms_Tpi_Dynamic_Row_Dto>> get_row_data_ptc(@PathVariable(
"vcode") String vcode,
803         @PathVariable("batch") String batch) {
804
805         var result = pms_serv.get_row_data(vcode, batch, "Power Tool cleaning");
806
807         return ResponseEntity.ok(result);
808     }
809
810     /**
811     * Get Row-Data for 'Blasting'
812     */
813     @GetMapping("/rd_blast/{vcode}/{batch}")
814     public ResponseEntity<List<Pms_Tpi_Dynamic_Row_Dto>> get_row_data_blast(@PathVariable(
"vcode") String vcode,

```

```

816         @PathVariable("batch") String batch) {
817
818         var result = pms_serv.get_row_data(vcode, batch, "Blasting to Sa 2.5");
819
820         return ResponseEntity.ok(result);
821     }
822
823
824     /**
825     * Get Row-Data for 'Prime' & Other Coats
826     */
827     @GetMapping("/rd_prime/{vcode}/{batch}/{calltype}")
828     public ResponseEntity<List<Pms_Tpi_Dynamic_Row_Dto>> get_row_data_prime_others(
829         @PathVariable("vcode") String vcode,
830         @PathVariable("batch") String batch, @PathVariable("calltype") String
831         calltype) {
832
833         var result = pms_serv.get_row_data(vcode, batch, calltype);
834
835         return ResponseEntity.ok(result);
836     }
837
838     /*****
839     /**
840     * Get sum of DU Qty by Combining all Call-Ref/Batch for All PO's
841     */
842     @GetMapping("/duSum")
843     public ResponseEntity<List<Pms_Root_Po_Du_Sum_Qty_Dto>> get_du_sum_qty_for_a_po(
844         Authentication authed_obj) {
845
846         var result = pms_serv.get_du_sum_qty_for_a_po(authed_obj.getName());
847
848         return ResponseEntity.ok(result);
849     }
850
851     /**
852     * Get PMS-FNL-QTY Data for the Vendor-Po
853     */
854     @GetMapping("/fnlQ")
855     @Secured({ "ROLE_OUTS_A", "ROLE_OUTS_S" })
856     public ResponseEntity<List<Pms_Fnl_Qty_Dto>> get_pms_fnl_qty_for_a_po(Authentication
857         authed_obj) {
858
859         List<Pms_Fnl_Qty_Dto> result = pms_serv.get_pms_fnl_qty_for_a_po(authed_obj.
860         getName());
861
862         return ResponseEntity.ok(result);
863     }
864
865     /*****
866     /**
867     * Post Data to 'Additional Data' for 'Power-Tool-Cleaning'
868     */
869     @PostMapping("/addI-ptc")
870     @Secured({ "ROLE_OUTS_A", "ROLE_OUTS_S" })
871     public ResponseEntity<Pms_Pow_tool_Vo> post_addi_inputs_by_vendor_for_ptc(
872         @RequestBody Pms_Pow_tool_Vo form_input,
873         Authentication authed_obj) throws Exception {
874
875         pms_serv.record_addi_info_ptc(form_input, authed_obj);
876     }
877

```

```

874         return ResponseEntity.ok(form_input);
875     }
876
877     /**
878     * Post Data to 'Additional Data' for 'Blasting-SA'
879     */
880     @PostMapping("/addI-blast")
881     @Secured({ "ROLE_OUTS_A", "ROLE_OUTS_S" })
882     public ResponseEntity<Pms_Blast_sa_2_5_Vo> post_addi_inputs_by_vendor_for_blasting(
883         @RequestBody Pms_Blast_sa_2_5_Vo form_input, Authentication authed_obj)
884         throws Exception {
885
886         pms_serv.record_addi_blast(form_input, authed_obj);
887
888         return ResponseEntity.ok(form_input);
889     }
890
891     /**
892     * Post Data to 'Additional Data' for 'Prime-coat' and 'Others' call-types
893     */
894     @PostMapping("/addI-pcoat")
895     @Secured({ "ROLE_OUTS_A", "ROLE_OUTS_S" })
896     public ResponseEntity<Pms_Prime_I_II_Vo> post_addi_inputs_by_vendor_for_prime_others(
897         @RequestBody Pms_Prime_I_II_Vo form_input, Authentication authed_obj) throws
898         Exception {
899
900         pms_serv.record_addi_prime_others(form_input, authed_obj);
901
902         return ResponseEntity.ok(form_input);
903     }
904
905     /**
906     * Post Data to 'Power-Tool-Cleaning' Stage
907     */
908     @PostMapping("/post_pow_tool/{status}")
909     @Secured({ "ROLE_OUTS_A", "ROLE_OUTS_S", "ROLE_INSA_D" })
910     public ResponseEntity<Pms_Pow_tool_Vo> post_power_tool_cleaning_stage_data(
911         @RequestBody Pms_Pow_tool_Vo form_input,
912         Authentication authed_obj, @PathVariable("status") String status) throws
913         Exception {
914
915         pms_serv.record_power_tool_cleaning_call(form_input, authed_obj, status);
916
917         return ResponseEntity.ok(form_input);
918     }
919
920     /**
921     * Post Data to 'Blast 2.5 Sa' Stage
922     */
923     @PostMapping("/blast_sa/{status}")
924     @Secured({ "ROLE_OUTS_A", "ROLE_OUTS_S", "ROLE_INSA_D" })
925     public ResponseEntity<Pms_Blast_sa_2_5_Vo> post_blast_sa_2_5(@RequestBody
926         Pms_Blast_sa_2_5_Vo form_input,
927         Authentication authed_obj, @PathVariable("status") String status) throws
928         Exception {
929
930         var saved_result = pms_serv.record_blast_2_5_all(form_input, authed_obj, status);
931
932         return ResponseEntity.ok(form_input);
933     }
934
935     /**

```

```

932     * Post Data to 'Prime I, II' Stage
933     */
934     @PostMapping("/prime-coat-I/{status}")
935     @Secured({ "ROLE_OUTS_A", "ROLE_OUTS_S", "ROLE_INSA_D" })
936     public ResponseEntity<Pms_Prime_I_II_Vo> post_prime_I_II(@RequestBody
937         Pms_Prime_I_II_Vo inter_coat,
938         Authentication authenticated_obj, @PathVariable("status") String status) throws
939         Exception {
940
941         pms_serv.record_inter_coat_I_II(inter_coat, authenticated_obj, status);
942
943         return ResponseEntity.ok(inter_coat);
944     }
945
946     /**
947     * Map Batch/Call_Ref_Num to a Call-Type
948     */
949     @PostMapping(value = "/mapB", consumes = { "multipart/form-data" })
950     @Secured({ "ROLE_OUTS_A", "ROLE_OUTS_S" })
951     public void map_batch_to_call_type(
952         // Batch/Call_Ref_Num
953         @RequestPart("batch") String batch,
954         // Call-Type
955         @RequestPart("call_type") String call_type,
956         // Paint-Type
957         @RequestPart("paint_type") String paint_type,
958         // Call-Request-Date-Time
959         @RequestPart("call_dt_tm") String call_dt_tm,
960         // TC
961         @RequestPart(value = "doc_p", required = false) MultipartFile doc_p,
962         // Authenticated Object
963         Authentication authenticated_obj) throws Exception {
964
965         System.out.println("-----> " + authenticated_obj.getName());
966         System.out.println("-----> " + batch);
967         System.out.println("-----> " + call_type);
968         System.out.println("-----> " + paint_type);
969         System.out.println("-----> " + call_dt_tm);
970         System.out.println("-----> " + doc_p);
971
972         pms_serv.record_batch_to_call_type(batch, call_type, paint_type, call_dt_tm,
973         doc_p, authenticated_obj);
974     }
975
976     /**
977     * Create PMS-Call Request/ Batch
978     */
979     @PostMapping(value = "/post_vreq", consumes = { "multipart/form-data" })
980     @Secured({ "ROLE_OUTS_A", "ROLE_OUTS_S" })
981     public void create_insp_call(
982         // Call-Type
983         @RequestPart("call_type") String call_type,
984         // Paint-Type
985         @RequestPart("paint_type") String paint_type,
986         // PO
987         @RequestPart("po_no") String po_no,
988         // Paint Scheme Reference Number
989         @RequestPart("paint_schm_ref") String paint_schm_ref,
990         // Project
991         @RequestPart("project") String project,
992         // WBS
993         @RequestPart("wo_no") String wo_no,
994         // Call-Request-Date-Time

```



```

993 //      @RequestParam("call_dt_tm") String call_dt_tm,
994 //      // TC for Paint
995 //      @RequestParam("doc_p") MultipartFile doc_file,
996 //      // Dynamic_Row_Data
997 //      @RequestParam("duCall") String duCall,
998 //      // Authenticated Object
999 //      Authentication authenticated_obj) {
1000
1001     System.out.println("-----> " + authenticated_obj.getName());
1002 //     System.out.println("-----> " + call_type);
1003 //     System.out.println("-----> " + paint_type);
1004     System.out.println("-----> " + po_no);
1005     System.out.println("-----> " + paint_schm_ref);
1006     System.out.println("-----> " + project);
1007     System.out.println("-----> " + wo_no);
1008 //     System.out.println("-----> " + call_dt_tm);
1009 //     System.out.println("-----> " + doc_file);
1010
1011     System.out.println("-----> " + duCall);
1012
1013     try {
1014         pms_serv.save_pms_call(po_no, paint_schm_ref, project, wo_no, duCall,
1015             authenticated_obj.getName());
1016     } catch (Exception e) {
1017         // TODO Auto-generated catch block
1018         e.printStackTrace();
1019     }
1020 }
1021
1022 /**
1023  * Call Report for a vendor
1024  */
1025 @GetMapping("/pmsVendRpt/{vcode}")
1026 @Secured({ "ROLE_OUTS_A", "ROLE_OUTS_S", "ROLE_INSA_A", "ROLE_INSA_D" })
1027 public ResponseEntity<List<Pms_Call_Report_Dto>> pms_call_report_for_a_vendor(
1028     @PathVariable("vcode") String vcode) {
1029
1030     List<Pms_Call_Report_Dto> result = null;
1031
1032     if (vcode.equals("all")) {
1033         result = pms_serv.pms_call_report_ALL_vendor();
1034     } else {
1035
1036         result = pms_serv.pms_call_report_for_a_vendor(vcode);
1037     }
1038
1039     return ResponseEntity.ok(result);
1040 }
1041
1042 /**
1043  * Completed/WIP Call-Type for a given given batch (Call History Tracking!!!)
1044  */
1045 @GetMapping("/hstryTrack/{vcode}")
1046 @Secured({ "ROLE_OUTS_A", "ROLE_OUTS_S" })
1047 public ResponseEntity<List<PMS_Batch_Track_Dto>> get_call_type_history_for_a_vendor(
1048     @PathVariable("vcode") String vcode) {
1049
1050     var result = pms_serv.get_call_type_history_for_a_vendor(vcode);
1051
1052     return ResponseEntity.ok(result);
1053 }
1054

```

```

1055
1056 /*****
1057 /
1058 /**
1059  * Get Completed call-list for printing & signing (FOR INSA)
1060  */
1061 @GetMapping("/compCalls")
1062 public ResponseEntity<List<PMS_Batch_Track_Dto>> get_completed_call_list() {
1063
1064     var result = pms_serv.get_completed_call_list();
1065
1066     return ResponseEntity.ok(result);
1067 }
1068
1069 /**
1070  * Get Completed Call-Type-Specfic-Data
1071  */
1072 @GetMapping("/callSpec/{vcode}/{inspYr}/{callNum}/{call_type}")
1073 public ResponseEntity<List<Pms_Tpi_Dynamic_Row_Dto>> get_completed_call_specifc_data(
1074     // Vendor Code
1075     @PathVariable("vcode") String vcode, @PathVariable("callNum") String callNum,
1076     @PathVariable("inspYr") String inspYr, @PathVariable("call_type") String
1077     call_type) {
1078
1079     List<Pms_Tpi_Dynamic_Row_Dto> result = null;
1080
1081     if (call_type.equals("Power Tool cleaning")) {
1082
1083         result = pms_serv.get_call_type_spacific_data_powr_tool_cleaning_serv(vcode,
1084             callNum, inspYr);
1085
1086     } else if (call_type.equals("Blasting to Sa 2.5")) {
1087
1088         result = pms_serv.get_call_type_spacific_data_blasting_serv(vcode, callNum,
1089             inspYr);
1090
1091     } else {
1092
1093         result = pms_serv.get_call_type_spacific_data_prime_others_serv(vcode, callNum
1094             , inspYr, call_type);
1095
1096     }
1097
1098     return ResponseEntity.ok(result);
1099 }
1100
1101 /**
1102  * Get Image Name & location for Each DU {For INSA}
1103  */
1104 @GetMapping("/imgLoc/{vcode}/{callNum}/{callType}")
1105 public ResponseEntity<List<Pms_Du_Img_GLoc_Dto>> get_image_name_location_data(
1106     // Vendor Code
1107     @PathVariable("vcode") String vcode,
1108     // Call Number
1109     @PathVariable("callNum") String callNum,
1110     // Call Type
1111     @PathVariable("callType") String callType) {
1112
1113     var result = pms_serv.get_image_name_location_data(vcode, Integer.parseInt(

```

```

        callNum), callType);
1113
1114     return ResponseEntity.ok(result);
1115 }
1116
1117 /**
1118  * Get Address/Profile Data for Vendor/User
1119  */
1120 @GetMapping("/vdet/{vcode}")
1121 public ResponseEntity<List<Address_Prime_Data_Dto>> get_vendor_profile(@PathVariable(
1122     "vcode") String vcode) {
1123
1124     var result = addr_serv.get_user_prime_data(vcode.substring(vcode.length() - 5));
1125
1126     return ResponseEntity.ok(result);
1127 }
1128
1129 package com.example.demo.model.dto.insp;
1130
1131 import com.example.demo.model.bo.insp.Pms_Geo_Img;
1132
1133 import lombok.AllArgsConstructor;
1134 import lombok.Builder;
1135 import lombok.Data;
1136 import lombok.NoArgsConstructor;
1137
1138 @NoArgsConstructor
1139 @AllArgsConstructor
1140 @Data
1141 @Builder
1142 public class Pms_Du_Img_GLoc_Dto {
1143
1144     private Integer duNum;
1145     private String img_name;
1146     private String lati;
1147     private String longi;
1148
1149     /**
1150      * Custom Constructor For DTO
1151      *
1152      * @param p
1153      */
1154     public Pms_Du_Img_GLoc_Dto(Pms_Geo_Img p) {
1155
1156         this.duNum = p.getPms_geo_pk().getDuNum();
1157         this.img_name = p.getImg_name();
1158         this.lati = p.getGeo_lat();
1159         this.longi = p.getGeo_long();
1160
1161     }
1162 }
1163
1164 package com.example.demo.model.bo.insp;
1165
1166 import jakarta.persistence.Column;
1167 import jakarta.persistence.EmbeddedId;
1168 import jakarta.persistence.Entity;
1169 import jakarta.persistence.Table;
1170 import lombok.AllArgsConstructor;
1171 import lombok.Builder;
1172 import lombok.Data;
1173 import lombok.NoArgsConstructor;
1174

```

```

1175 @NoArgsConstructor
1176 @AllArgsConstructor
1177 @Data
1178 @Builder
1179 @Entity
1180 @Table(name = "PMS_FNL_QTY")
1181 public class Pms_Fnl_Qty {
1182
1183     @EmbeddedId
1184     private Pms_Fnl_Qty_PK pms_fnl_pk;
1185
1186     /**
1187      * So-Far Accepted Quantity for the PO-DU_NUM
1188      */
1189     @Column(name = "ACC_QTY")
1190     private Double accQty;
1191
1192     @Column(name = "PMS_CALL_YR", length = 4)
1193     private String pmsCallYr;
1194 }
1195
1196 package com.example.demo.dao.insp;
1197
1198 import java.util.List;
1199
1200 import org.springframework.data.jpa.repository.JpaRepository;
1201 import org.springframework.data.jpa.repository.Query;
1202 import org.springframework.data.repository.query.Param;
1203 import org.springframework.stereotype.Repository;
1204
1205 import com.example.demo.model.bo.insp.Pms_Fnl_Qty;
1206 import com.example.demo.model.bo.insp.Pms_Fnl_Qty_PK;
1207 import com.example.demo.model.dto.insp.Pms_Fnl_Qty_Dto;
1208
1209 @Repository
1210 public interface Pms_Fnl_Qty_Dao extends JpaRepository<Pms_Fnl_Qty, Pms_Fnl_Qty_PK> {
1211
1212     @Query("""
1213         SELECT
1214             new com.example.demo.model.dto.insp.Pms_Fnl_Qty_Dto(f)
1215         FROM
1216             Pms_Fnl_Qty f
1217         WHERE
1218             f.pms_fnl_pk.vcode = :vcode
1219         """)
1220     List<Pms_Fnl_Qty_Dto> get_pms_fnl_qty_for_a_po(@Param("vcode") String vcode);
1221
1222 }
1223
1224 package com.example.demo.model.dto.insp;
1225
1226 import com.example.demo.model.bo.insp.Pms_Fnl_Qty;
1227
1228 import lombok.AllArgsConstructor;
1229 import lombok.Builder;
1230 import lombok.Data;
1231 import lombok.NoArgsConstructor;
1232
1233 @NoArgsConstructor
1234 @AllArgsConstructor
1235 @Data
1236 @Builder
1237 public class Pms_Fnl_Qty_Dto {

```

```

1239
1240 // Flattened ID fields
1241 private String po;
1242 private Integer poSno;
1243 private String vcode;
1244 private String pmsStage;
1245
1246 // Entity fields
1247 private Double accQty;
1248 private String pmsCallYr;
1249
1250 // Constructor to map from entity to DTO
1251 public Pms_Fnl_Qty_Dto(Pms_Fnl_Qty entity) {
1252     this.po = entity.getPms_fnl_pk().getPo();
1253     this.poSno = entity.getPms_fnl_pk().getPoSno();
1254     this.vcode = entity.getPms_fnl_pk().getVcode();
1255
1256     this.accQty = entity.getAccQty();
1257     this.pmsCallYr = entity.getPmsCallYr();
1258 }
1259 }
1260
1261 package com.example.demo.model.bo.insp;
1262
1263 import java.io.Serializable;
1264
1265 import jakarta.persistence.Column;
1266 import jakarta.persistence.Embeddable;
1267 import lombok.AllArgsConstructor;
1268 import lombok.Builder;
1269 import lombok.Data;
1270 import lombok.NoArgsConstructor;
1271
1272 @NoArgsConstructor
1273 @AllArgsConstructor
1274 @Data
1275 @Embeddable
1276 @Builder
1277 public class Pms_Fnl_Qty_PK implements Serializable {
1278
1279     @Column(name = "VCODE", length = 10)
1280     private String vcode;
1281
1282     @Column(name = "PO", length = 10)
1283     private String po;
1284
1285     @Column(name = "DU_NUM", precision = 10)
1286     private Integer poSno;
1287
1288 }
1289
1290 package com.example.demo.model.bo.insp;
1291
1292 import java.time.LocalDateTime;
1293
1294 import jakarta.persistence.Column;
1295 import jakarta.persistence.EmbeddedId;
1296 import jakarta.persistence.Entity;
1297 import jakarta.persistence.Table;
1298 import lombok.AllArgsConstructor;
1299 import lombok.Builder;
1300 import lombok.Data;
1301 import lombok.NoArgsConstructor;
1302

```

```

1303 @NoArgsConstructor
1304 @AllArgsConstructor
1305 @Data
1306 @Builder
1307 @Entity
1308 @Table(name = "PMS_GEO_IMG")
1309 public class Pms_Geo_Img {
1310
1311     @EmbeddedId
1312     private Pms_Geo_Img_PK pms_geo_pk;
1313
1314     @Column(name = "GEO_LATI")
1315     private String geo_lat;
1316
1317     @Column(name = "GEO_LONG")
1318     private String geo_long;
1319
1320     @Column(name = "img_name")
1321     private String img_name;
1322
1323     @Column(name = "fcm_token")
1324     private String fcm_token;
1325
1326     @Column(name = "uuid")
1327     private String uuid;
1328
1329     @Column(name = "UPD_ON")
1330     private LocalDateTime updOn;
1331
1332     @Column(name = "upd_by")
1333     private String upd_by;
1334 }
1335
1336 package com.example.demo.dao.insp;
1337
1338 import java.util.List;
1339
1340 import org.springframework.data.jpa.repository.JpaRepository;
1341 import org.springframework.data.jpa.repository.Query;
1342 import org.springframework.stereotype.Repository;
1343
1344 import com.example.demo.model.bo.insp.Pms_Geo_Img;
1345 import com.example.demo.model.bo.insp.Pms_Geo_Img_PK;
1346 import com.example.demo.model.dto.insp.Pms_Du_Img_GLoc_Dto;
1347
1348 @Repository
1349 public interface Pms_Geo_Img_Dao extends JpaRepository<Pms_Geo_Img, Pms_Geo_Img_PK> {
1350
1351     /**
1352      * Get Image Name & location for Each DU
1353      *
1354      * @param vcode
1355      * @param duNum
1356      * @param callType
1357      * @return
1358      */
1359     @Query("""
1360         SELECT
1361             new com.example.demo.model.dto.insp.Pms_Du_Img_GLoc_Dto(p)
1362         FROM
1363             Pms_Geo_Img p
1364         WHERE
1365             p.pms_geo_pk.vcode = :vcode
1366             AND p.pms_geo_pk.callNum = :callNum

```

```

1367         AND p.pms_geo_pk.callType = :callType
1368     ORDER BY
1369         p.pms_geo_pk.poNum, p.pms_geo_pk.duNum ASC
1370     """)
1371     List<Pms_Du_Img_GLoc_Dto> get_image_name_location_data(String vcode, int callNum,
1372         String callType);
1373 }
1374
1375 package com.example.demo.model.bo.insp;
1376
1377 import java.io.Serializable;
1378 import java.time.LocalDateTime;
1379
1380 import jakarta.persistence.Column;
1381 import jakarta.persistence.Embeddable;
1382 import lombok.AllArgsConstructor;
1383 import lombok.Builder;
1384 import lombok.Data;
1385 import lombok.NoArgsConstructor;
1386
1387 @NoArgsConstructor
1388 @AllArgsConstructor
1389 @Data
1390 @Builder
1391 @Embeddable
1392 public class Pms_Geo_Img_PK implements Serializable {
1393
1394     @Column(name = "vcode", length = 10)
1395     private String vcode;
1396
1397     @Column(name = "call_num", precision = 10)
1398     private Integer callNum;
1399
1400     @Column(name = "poNum", length = 150)
1401     private String poNum;
1402
1403     @Column(name = "duNum", precision = 10)
1404     private Integer duNum;
1405
1406     @Column(name = "call_Type", length = 150)
1407     private String callType;
1408
1409     @Column(name = "call_Yr")
1410     private String call_Yr;
1411 }
1412
1413 package com.example.demo.model.bo.insp;
1414
1415 import java.math.BigDecimal;
1416
1417 import jakarta.persistence.Column;
1418 import jakarta.persistence.Entity;
1419 import jakarta.persistence.Id;
1420 import jakarta.persistence.Table;
1421 import lombok.AllArgsConstructor;
1422 import lombok.Data;
1423 import lombok.NoArgsConstructor;
1424
1425 @NoArgsConstructor
1426 @AllArgsConstructor
1427 @Data
1428 @Entity
1429 @Table(name = "PMS_PAINT_CURING_TIME")

```

```

1430 public class PMS_Paint_Curing_Time {
1431
1432     @Id
1433     @Column(name = "paint_type", length = 200)
1434     private String paint_type;
1435
1436     @Column(name = "CURING_TIME")
1437     private BigDecimal curing_time;
1438 }
1439
1440 package com.example.demo.dao.insp;
1441
1442 import org.springframework.data.jpa.repository.JpaRepository;
1443 import org.springframework.stereotype.Repository;
1444
1445 import com.example.demo.model.bo.insp.PMS_Paint_Curing_Time;
1446
1447 @Repository
1448 public interface PMS_Paint_Curing_Time_Dao extends JpaRepository<PMS_Paint_Curing_Time,
String> {
1449
1450 }
1451
1452 package com.example.demo.model.bo.insp;
1453
1454 import java.math.BigDecimal;
1455 import java.util.Date;
1456
1457 import com.example.demo.model.bo.outs.Po_Details;
1458 import com.example.demo.model.bo.outs.Po_Details_PK;
1459
1460 import jakarta.persistence.Column;
1461 import jakarta.persistence.EmbeddedId;
1462 import jakarta.persistence.Entity;
1463 import jakarta.persistence.Table;
1464 import jakarta.persistence.Temporal;
1465 import jakarta.persistence.TemporalType;
1466 import lombok.AllArgsConstructor;
1467 import lombok.Data;
1468 import lombok.NoArgsConstructor;
1469
1470 @NoArgsConstructor
1471 @AllArgsConstructor
1472 @Data
1473 @Entity
1474 @Table(name = "PMS_PO_Details")
1475 public class PMS_Po_Details {
1476
1477     @EmbeddedId
1478     private PMS_Po_Details_PK po_det_pk;
1479
1480     @Column(name = "ZZ_PO_DT")
1481     @Temporal(TemporalType.DATE)
1482     private Date zzPoDt;
1483
1484     @Column(name = "ZZ_WBS", precision = 8)
1485     private Long zzWbs;
1486
1487     @Column(name = "ZZ_PO_SLNR", precision = 5)
1488     private Integer zzPoSlnr;
1489
1490     @Column(name = "ZZ_DU_CODE", length = 18)
1491     private String zzDuCode;
1492

```



```
1493 @Column(name = "ZZ_UOM", length = 3)
1494 private String zzUom;
1495
1496 @Column(name = "ZZ_DESC", length = 40)
1497 private String zzDesc;
1498
1499 @Column(name = "ZZ_QTY", precision = 13, scale = 3)
1500 private BigDecimal zzQty;
1501
1502 @Column(name = "ZZ_WT", precision = 13, scale = 3)
1503 private BigDecimal zzWt;
1504
1505 @Column(name = "ZZ_RSP", length = 10)
1506 private String zzRsp;
1507
1508 @Column(name = "ZZ_RSS", length = 10)
1509 private String zzRss;
1510
1511 @Column(name = "ZZ_SURFAREA", precision = 13, scale = 3)
1512 private BigDecimal zzSurfarea;
1513
1514 @Column(name = "ZZ_DESCWBS", length = 24)
1515 private String zzDescwbs;
1516
1517 @Column(name = "ZZ_TM", length = 7)
1518 private String zzTm;
1519
1520 @Column(name = "FABHARGE", length = 20)
1521 private String fabharge;
1522
1523 @Column(name = "PACHARGE", length = 20)
1524 private String pacharge;
1525
1526 @Column(name = "QCP", length = 30)
1527 private String qcp;
1528
1529 @Column(name = "INSCAT", length = 1)
1530 private String inscat;
1531
1532 @Column(name = "NAME1", length = 40)
1533 private String name1;
1534
1535 @Column(name = "PCF", length = 4)
1536 private String pcf;
1537
1538 @Column(name = "PAREA", length = 20)
1539 private String parea;
1540
1541 @Column(name = "RSF", length = 10)
1542 private String rsf;
1543
1544 @Column(name = "DEL_IND", columnDefinition = "CHAR(1 BYTE)")
1545 private String delInd;
1546
1547 @Column(name = "DELY_DT")
1548 @Temporal(TemporalType.DATE)
1549 private Date delyDt;
1550
1551 @Column(name = "ZZ_LGORT", length = 4)
1552 private String zzLgort;
1553
1554 @Column(name = "SB_VCODE", length = 10)
1555 private String sbVcode;
1556
```

```

1557     @Column(name = "HT_VCODE", length = 10)
1558     private String htVcode;
1559
1560     @Column(name = "GV_VCODE", length = 10)
1561     private String gvVcode;
1562
1563     @Column(name = "PO_DIVERT", columnDefinition = "CHAR(1 BYTE)")
1564     private String poDivert;
1565
1566     @Column(name = "EXTRAS_VAL", precision = 13, scale = 2)
1567     private BigDecimal extrasVal;
1568
1569     @Column(name = "FREIGHT", precision = 13, scale = 2)
1570     private BigDecimal freight;
1571
1572     @Column(name = "CUST", length = 4)
1573     private String cust;
1574
1575     @Column(name = "ZZWBS1", length = 24)
1576     private String zzwbs1;
1577
1578     @Column(name = "DRG_NO", length = 25)
1579     private String drgNo;
1580
1581     @Column(name = "ENQUIRY", length = 30)
1582     private String enquiry;
1583
1584 }
1585
1586 package com.example.demo.dao.insp;
1587
1588 import java.util.List;
1589
1590 import org.springframework.data.jpa.repository.JpaRepository;
1591 import org.springframework.data.jpa.repository.Query;
1592 import org.springframework.stereotype.Repository;
1593
1594 import com.example.demo.model.bo.insp.PMS_Po_Details;
1595 import com.example.demo.model.bo.insp.PMS_Po_Details_PK;
1596 import com.example.demo.model.dto.insp.Pms_Po_Details_Dto;
1597
1598 @Repository
1599 public interface PMS_Po_Details_Dao extends JpaRepository<PMS_Po_Details,
1600 PMS_Po_Details_PK> {
1601
1602     /**
1603      * Get All PO's belongs to Vendor-Code
1604      *
1605      * @param vcode
1606      * @return
1607      */
1608     @Query("""
1609         SELECT
1610             new com.example.demo.model.dto.insp.Pms_Po_Details_Dto(p)
1611         FROM
1612             PMS_Po_Details p
1613         WHERE
1614             p.po_det_pk.zzShotVendor = :vcode
1615         """)
1616     List<Pms_Po_Details_Dto> get_po_det_by_vendor(String vcode);
1617
1618     /**
1619      * PO Details as first & sb-vendor
1620      *

```

```

1620     * @param vcode
1621     * @return
1622     */
1623     @Query("""
1624         SELECT
1625             new com.example.demo.model.dto.insp.Pms_Po_Details_Dto(p)
1626         FROM
1627             PMS_Po_Details p
1628         WHERE
1629             p.sbVcode = :vcode
1630         """)
1631     List<Pms_Po_Details_Dto> get_po_for_2_nd_vendor(String vcode);
1632 }
1633
1634 package com.example.demo.model.dto.insp;
1635
1636 import java.math.BigDecimal;
1637 import java.util.Date;
1638
1639 import com.example.demo.model.bo.insp.PMS_Po_Details;
1640
1641 import lombok.AllArgsConstructor;
1642 import lombok.Data;
1643 import lombok.EqualsAndHashCode;
1644 import lombok.NoArgsConstructor;
1645
1646 @NoArgsConstructor
1647 @AllArgsConstructor
1648 @Data
1649 @EqualsAndHashCode
1650 public class Pms_Po_Details_Dto {
1651
1652     /**
1653      * The goal is to flatten(!) the embedded 'Po_Details_PK' object into its own
1654      * fields directly inside the 'Po_Details' object.
1655      */
1656
1657     // Po_Details fields
1658     private Date zzPoDt;
1659     private Long zzWbs;
1660     private Integer zzPoSlr;
1661     private String zzDuCode;
1662     private String zzUom;
1663     private String zzDesc;
1664     private BigDecimal zzQty;
1665     private BigDecimal zzWt;
1666     private String zzRsp;
1667     private String zzRss;
1668     private BigDecimal zzSurfarea;
1669     private String zzDescwbs;
1670     private String zzTm;
1671     private String fabharge;
1672     private String pacharge;
1673     private String qcp;
1674     private String inscat;
1675     private String name1;
1676     private String pcf;
1677     private String parea;
1678     private String rsf;
1679     private String delInd;
1680     private Date delyDt;
1681     private String zzLgort;
1682     private String sbVcode;
1683     private String htVcode;

```

```

1684 private String gvVcode;
1685 private String poDivert;
1686 private BigDecimal extrasVal;
1687 private BigDecimal freight;
1688 private String cust;
1689 private String zzwbs1;
1690 private String drgNo;
1691 private String enquiry;
1692
1693 // Fields from Po_Details_PK
1694 private String zzEbeln;
1695 private String zzPlant;
1696 private String zzPurGrp;
1697 private Integer zzEbelp;
1698 private String zzShotVendor;
1699
1700 /**
1701  * Never forget to define it!!
1702  *
1703  * Custom Constructor for Getting Actual-BO as 'Single' input
1704  */
1705 public Pms_Po_Details_Dto(PMS_Po_Details p) {
1706
1707     // From Po_Details
1708     this.zzPoDt = p.getZzPoDt();
1709     this.zzWbs = p.getZzWbs();
1710     this.zzPoSlr = p.getZzPoSlr();
1711     this.zzDuCode = p.getZzDuCode();
1712     this.zzUom = p.getZzUom();
1713     this.zzDesc = p.getZzDesc();
1714     this.zzQty = p.getZzQty();
1715     this.zzWt = p.getZzWt();
1716     this.zzRsp = p.getZzRsp();
1717     this.zzRss = p.getZzRss();
1718     this.zzSurfarea = p.getZzSurfarea();
1719     this.zzDescwbs = p.getZzDescwbs();
1720     this.zzTm = p.getZzTm();
1721     this.fabcharge = p.getFabcharge();
1722     this.pacharge = p.getPacharge();
1723     this.qcp = p.getQcp();
1724     this.inscat = p.getInscat();
1725     this.name1 = p.getName1();
1726     this.pcf = p.getPcf();
1727     this.parea = p.getParea();
1728     this.rsfc = p.getRsfc();
1729     this.delInd = p.getDelInd();
1730     this.delyDt = p.getDelyDt();
1731     this.zzLgort = p.getZzLgort();
1732     this.sbVcode = p.getSbVcode();
1733     this.htVcode = p.getHtVcode();
1734     this.gvVcode = p.getGvVcode();
1735     this.poDivert = p.getPoDivert();
1736     this.extrasVal = p.getExtrasVal();
1737     this.freight = p.getFreight();
1738     this.cust = p.getCust();
1739     this.zzwbs1 = p.getZzwbs1();
1740     this.drgNo = p.getDrgNo();
1741     this.enquiry = p.getEnquiry();
1742
1743     // From Po_Details_PK
1744     if (p.getPo_det_pk() != null) {
1745         this.zzEbeln = p.getPo_det_pk().getZzEbeln();
1746         this.zzPlant = p.getPo_det_pk().getZzPlant();
1747         this.zzPurGrp = p.getPo_det_pk().getZzPurGrp();

```

```

1748         this.zzEbelp = p.getPo_det_pk().getZzEbelp();
1749         this.zzShotVendor = p.getPo_det_pk().getZzShotVendor();
1750     }
1751 }
1752
1753 }
1754
1755 package com.example.demo.model.bo.insp;
1756
1757 import java.io.Serializable;
1758
1759 import jakarta.persistence.Column;
1760 import jakarta.persistence.Embeddable;
1761 import lombok.AllArgsConstructor;
1762 import lombok.Data;
1763 import lombok.NoArgsConstructor;
1764
1765 @NoArgsConstructor
1766 @AllArgsConstructor
1767 @Data
1768 @Embeddable
1769 public class PMS_Po_Details_PK implements Serializable {
1770
1771     @Column(name = "ZZ_EBELN", length = 10)
1772     private String zzEbelp;
1773
1774     @Column(name = "ZZ_PLANT", length = 4)
1775     private String zzPlant;
1776
1777     @Column(name = "ZZ_PUR_GRP", length = 3)
1778     private String zzPurGrp;
1779
1780     @Column(name = "ZZ_EBELP", precision = 5)
1781     private Integer zzEbelp;
1782
1783     @Column(name = "ZZ_SHOT_VENDOR", length = 10)
1784     private String zzShotVendor;
1785 }
1786
1787 package com.example.demo.model.bo.insp;
1788
1789 import java.math.BigDecimal;
1790 import java.time.LocalDate;
1791 import java.time.LocalDateTime;
1792
1793 import jakarta.persistence.Column;
1794 import jakarta.persistence.EmbeddedId;
1795 import jakarta.persistence.Entity;
1796 import jakarta.persistence.Table;
1797 import lombok.AllArgsConstructor;
1798 import lombok.Builder;
1799 import lombok.Data;
1800 import lombok.NoArgsConstructor;
1801
1802 @NoArgsConstructor
1803 @AllArgsConstructor
1804 @Data
1805 @Builder
1806 @Entity
1807 @Table(name = "PMS_POW_TOOL")
1808 public class Pms_Pow_Tool {
1809
1810     /* ----- */
1811

```

```

1812 @EmbeddedId
1813 private Pms_Pow_Tool_PK pow_tool_pk;
1814
1815 /* ----- */
1816
1817 private String duDesc;
1818 private Double duCallQty;
1819
1820 @Column(name = "DRY_BULB_T", precision = 6, scale = 3)
1821 private BigDecimal dryBulbT;
1822
1823 @Column(name = "WET_BULB_T", precision = 6, scale = 3)
1824 private BigDecimal wetBulbT;
1825
1826 @Column(name = "DEW_PT", precision = 6, scale = 3)
1827 private BigDecimal dewPt;
1828
1829 private BigDecimal rh;
1830
1831 private LocalDate readDt;
1832 private LocalTime readTimeSt;
1833 private LocalTime readTimeEnd;
1834
1835 @Column(length = 250)
1836 private String inspRmrk;
1837
1838 /* ----- */
1839
1840 @Column(name = "clean_by")
1841 private String cleanBy;
1842 private String rawRustGrade;
1843 @Column(name = "Dust_Free_Check")
1844 private String dust_free_check_result;
1845 private String visInsp;
1846
1847 /* ----- */
1848
1849 @Column(length = 7)
1850 private String inspClrBy;
1851
1852 private LocalDate inspClrDt;
1853
1854 }
1855
1856 package com.example.demo.dao.insp;
1857
1858 import java.util.List;
1859
1860 import org.springframework.data.jpa.repository.JpaRepository;
1861 import org.springframework.data.jpa.repository.Query;
1862 import org.springframework.stereotype.Repository;
1863
1864 import com.example.demo.model.bo.insp.Pms_Pow_Tool;
1865 import com.example.demo.model.bo.insp.Pms_Pow_Tool_PK;
1866 import com.example.demo.model.dto.insp.Pms_Tpi_Dynamic_Row_Dto;
1867
1868 @Repository
1869 public interface Pms_Pow_Tool_Dao extends JpaRepository<Pms_Pow_Tool, Pms_Pow_Tool_PK> {
1870
1871     /**
1872      * Dynamic_Row_Data_For_A_Vendor_For_A_Call_Number_OR_Batch for
1873      * 'Power-Tool-Cleaning'
1874      */
1875     @Query("""

```

```

1876         SELECT
1877             new com.example.demo.model.dto.insp.Pms_Tpi_Dynamic_Row_Dto(p, q)
1878         FROM
1879             Pms_Pow_Tool p
1880         LEFT JOIN
1881             Po_Details q
1882             ON p.pow_tool_pk.poNum = q.po_det_pk.zzEbeln
1883             AND p.pow_tool_pk.duNum = q.po_det_pk.zzEbelp
1884         WHERE
1885             p.pow_tool_pk.vcode = :vcode
1886             AND p.pow_tool_pk.callRefNum = :call_num
1887         """)
1888     public List<Pms_Tpi_Dynamic_Row_Dto> get_dynamic_row_data(String vcode, Integer
call_num);
1889 }
1890
1891 package com.example.demo.model.dto.insp;
1892
1893 import java.math.BigDecimal;
1894 import java.time.LocalDate;
1895 import java.time.LocalTime;
1896
1897 import com.example.demo.model.bo.insp.Pms_Pow_Tool;
1898 import com.google.auto.value.AutoValue.Builder;
1899
1900 import lombok.AllArgsConstructor;
1901 import lombok.Data;
1902 import lombok.NoArgsConstructor;
1903
1904 @NoArgsConstructor
1905 @AllArgsConstructor
1906 @Data
1907 @Builder
1908 public class Pms_Pow_Tool_Dto {
1909
1910     // Flattened fields from Pms_Pow_Tool_PK
1911     private Integer callRefNum;
1912     private String vcode;
1913     private String poNum;
1914     private Integer duNum;
1915
1916     // Other fields from Pms_Pow_Tool
1917     private String cleanBy;
1918     private String rawRustGrade;
1919     private BigDecimal dryBulbT;
1920     private BigDecimal wetBulbT;
1921     private BigDecimal dewPt;
1922     private BigDecimal rh;
1923     private LocalDate readDt;
1924     private LocalTime readTimeSt;
1925     private LocalTime readTimeEnd;
1926     private String inspClrBy;
1927     private LocalDate inspClrDt;
1928     private String inspRmrk;
1929
1930     // Constructor that takes a single Pms_Pow_Tool entity object
1931     public Pms_Pow_Tool_Dto(Pms_Pow_Tool pmsPowTool) {
1932         this.callRefNum = pmsPowTool.getPow_tool_pk().getCallRefNum();
1933         this.vcode = pmsPowTool.getPow_tool_pk().getVcode();
1934         this.poNum = pmsPowTool.getPow_tool_pk().getPoNum();
1935         this.duNum = pmsPowTool.getPow_tool_pk().getDuNum();
1936         this.cleanBy = pmsPowTool.getCleanBy();
1937         this.rawRustGrade = pmsPowTool.getRawRustGrade();
1938         this.dryBulbT = pmsPowTool.getDryBulbT();

```

```

1939         this.wetBulbT = pmsPowTool.getWetBulbT();
1940         this.dewPt = pmsPowTool.getDewPt();
1941         this.rh = pmsPowTool.getRh();
1942         this.readDt = pmsPowTool.getReadDt();
1943         this.readTimeSt = pmsPowTool.getReadTimeSt();
1944         this.readTimeEnd = pmsPowTool.getReadTimeEnd();
1945         this.inspClrBy = pmsPowTool.getInspClrBy();
1946         this.inspClrDt = pmsPowTool.getInspClrDt();
1947         this.inspRmrk = pmsPowTool.getInspRmrk();
1948     }
1949 }
1950
1951 package com.example.demo.model.bo.insp;
1952
1953 import java.io.Serializable;
1954
1955 import jakarta.persistence.Column;
1956 import jakarta.persistence.Embeddable;
1957 import lombok.AllArgsConstructor;
1958 import lombok.Builder;
1959 import lombok.Data;
1960 import lombok.NoArgsConstructor;
1961
1962 @NoArgsConstructor
1963 @AllArgsConstructor
1964 @Data
1965 @Builder
1966 @Embeddable
1967 public class Pms_Pow_Tool_PK implements Serializable {
1968
1969     @Column(name = "CALL_REF_NUM", precision = 10)
1970     private Integer callRefNum;
1971
1972     @Column(name = "VCODE", length = 10)
1973     private String vcode;
1974
1975     @Column(name = "PO_NUM", length = 10)
1976     private String poNum;
1977
1978     @Column(name = "DU_NUM", precision = 5)
1979     private Integer duNum;
1980 }
1981
1982 package com.example.demo.model.vo.insp;
1983
1984 import java.util.List;
1985
1986 import com.fasterxml.jackson.annotation.JsonProperty;
1987 import com.google.auto.value.AutoValue.Builder;
1988
1989 import lombok.AllArgsConstructor;
1990 import lombok.Data;
1991 import lombok.NoArgsConstructor;
1992
1993 @NoArgsConstructor
1994 @AllArgsConstructor
1995 @Data
1996 @Builder
1997 public class Pms_Pow_tool_Vo {
1998
1999     /*****
2000
2001     @JsonProperty("frm_grp_ctrl_vcode")
2002     private String frmGrpCtrlVcode;

```



```

2003
2004     @JsonProperty("frm_grp_ctrl_ref")
2005     private Integer frmGrpCtrlRef;
2006
2007     @JsonProperty("frm_grp_ctrl_po")
2008     private String frmGrpCtrlPo;
2009
2010     @JsonProperty("frm_grp_ctrl_call_type")
2011     private String frm_grp_ctrl_call_type;
2012
2013     @JsonProperty("frm_grp_ctrl_paint_type")
2014     private String frm_grp_ctrl_paint_type;
2015
2016     /*****/
2017
2018     @JsonProperty("clean_used")
2019     private String clean_used;
2020
2021     @JsonProperty("dust_free_check_result")
2022     private String dust_free_check_result;
2023
2024     @JsonProperty("raw_matl_rust_grade")
2025     private String raw_matl_rust_grade;
2026
2027     @JsonProperty("visual_insp")
2028     private String visual_insp;
2029
2030     /*****/
2031
2032     @JsonProperty("row_form_group")
2033     private List<Pms_Pow_tool_Vo_Row_Group> rowFormGroup;
2034
2035     /*****/
2036 }
2037
2038 package com.example.demo.model.vo.insp;
2039
2040 import java.math.BigDecimal;
2041 import java.time.LocalDate;
2042 import java.time.LocalDateTime;
2043
2044 import com.fasterxml.jackson.annotation.JsonProperty;
2045 import com.google.auto.value.AutoValue.Builder;
2046
2047 import lombok.AllArgsConstructor;
2048 import lombok.Data;
2049 import lombok.NoArgsConstructor;
2050
2051 @NoArgsConstructor
2052 @AllArgsConstructor
2053 @Data
2054 @Builder
2055 public class Pms_Pow_tool_Vo_Row_Group {
2056
2057     private Integer du;
2058
2059     @JsonProperty("du_description")
2060     private String duDescription;
2061
2062     private Double quantity;
2063
2064     @JsonProperty("dry_bulb_temp")
2065     private BigDecimal dryBulbTemp;
2066

```

```

2067     @JsonProperty("wet_bulb_temp")
2068     private BigDecimal wetBulbTemp;
2069
2070     @JsonProperty("dew_point")
2071     private BigDecimal dewPoint;
2072
2073     private BigDecimal rh;
2074
2075     @JsonProperty("date")
2076     private LocalDate reading_date;
2077
2078     @JsonProperty("start_time")
2079     private LocalTime startTime;
2080
2081     @JsonProperty("end_time")
2082     private LocalTime endTime;
2083
2084     private String remark;
2085
2086     private BigDecimal extra;
2087
2088 }
2089
2090 package com.example.demo.model.bo.insp;
2091
2092 import java.math.BigDecimal;
2093 import java.time.LocalDate;
2094 import java.time.LocalTime;
2095
2096 import jakarta.persistence.Column;
2097 import jakarta.persistence.EmbeddedId;
2098 import jakarta.persistence.Entity;
2099 import jakarta.persistence.Table;
2100 import lombok.AllArgsConstructor;
2101 import lombok.Builder;
2102 import lombok.Data;
2103 import lombok.NoArgsConstructor;
2104
2105 @NoArgsConstructor
2106 @AllArgsConstructor
2107 @Data
2108 @Builder
2109 @Entity
2110 @Table(name = "PMS_Pirme_Coat")
2111 public class Pms_Prime_I_II {
2112
2113     /* ----- */
2114
2115     @EmbeddedId
2116     private Pms_Prime_I_II_PK pow_inter_coat_pk;
2117
2118     /* ----- */
2119
2120     private String duDesc;
2121     private Double duCallQty;
2122
2123     @Column(name = "DRY_BULB_T", precision = 6, scale = 3)
2124     private BigDecimal dryBulbT;
2125
2126     @Column(name = "WET_BULB_T", precision = 6, scale = 3)
2127     private BigDecimal wetBulbT;
2128
2129     @Column(name = "MAT_SURF_T", precision = 6, scale = 3)
2130     private BigDecimal matelSurfT;

```

```
2131
2132     @Column(name = "DEW_PT", precision = 6, scale = 3)
2133     private BigDecimal dewPt;
2134
2135     private BigDecimal rh;
2136
2137     private LocalDate readDt;
2138     private LocalTime readTimeSt;
2139     private LocalTime readTimeEnd;
2140
2141     @Column(length = 250)
2142     private String inspRmrk;
2143
2144     /* ----- */
2145
2146     @Column(name = "paint_spec")
2147     private String paint_spec;
2148
2149     @Column(name = "paint_shade")
2150     private String paint_shade;
2151
2152     @Column(name = "paint_manuf")
2153     private String paint_manuf;
2154
2155     @Column(name = "supplier_trader")
2156     private String supplier_trader;
2157
2158     @Column(name = "manuf_dt")
2159     private LocalDate manuf_dt;
2160
2161     @Column(name = "manuf_batch")
2162     private String manuf_batch;
2163
2164     @Column(name = "manuf_tc")
2165     private String manuf_tc;
2166
2167     @Column(name = "paint_exp_dt")
2168     private LocalDate paint_exp_dt;
2169
2170     @Column(name = "mix_ratio")
2171     private String mix_ratio;
2172
2173     @Column(name = "num_coat")
2174     private String num_coat;
2175
2176     @Column(name = "time_of_mix")
2177     private String time_of_mix;
2178
2179     @Column(name = "wet_film_thick")
2180     private String wet_film_thick;
2181
2182     @Column(name = "spary_gun")
2183     private String spary_gun;
2184
2185     @Column(name = "hard_dry_time_req")
2186     private String hard_dry_time_req;
2187
2188     @Column(name = "dft_req")
2189     private String dft_req;
2190
2191     @Column(name = "dft_act")
2192     private String dft_act;
2193
2194     @Column(name = "visual_insp")
```

```

2195     private String visual_insp;
2196
2197     @Column(name = "cross_cut_dft_50")
2198     private String cross_cut_dft_50;
2199
2200     @Column(name = "cross_cut_dft_50_125")
2201     private String cross_cut_dft_50_125;
2202
2203     @Column(name = "cross_cut_dft_125")
2204     private String cross_cut_dft_125;
2205
2206     /* ----- */
2207
2208     @Column(length = 7)
2209     private String inspClrBy;
2210
2211     private LocalDate inspClrDt;
2212
2213 }
2214
2215 package com.example.demo.dao.insp;
2216
2217 import java.util.List;
2218
2219 import org.springframework.data.jpa.repository.JpaRepository;
2220 import org.springframework.data.jpa.repository.Query;
2221 import org.springframework.stereotype.Repository;
2222
2223 import com.example.demo.model.bo.insp.Pms_Prime_I_II;
2224 import com.example.demo.model.bo.insp.Pms_Prime_I_II_PK;
2225 import com.example.demo.model.dto.insp.Pms_Tpi_Dynamic_Row_Dto;
2226
2227 @Repository
2228 public interface Pms_Prime_I_II_Dao extends JpaRepository<Pms_Prime_I_II,
Pms_Prime_I_II_PK> {
2229
2230     /**
2231      * Dynamic_Row_Data_For_A_Vendor_For_A_Call_Number_OR_Batch for 'Others'
2232      */
2233     @Query("""
2234         SELECT
2235             new com.example.demo.model.dto.insp.Pms_Tpi_Dynamic_Row_Dto(p,q)
2236         FROM
2237             Pms_Prime_I_II p
2238         LEFT JOIN
2239             Po_Details q
2240             ON p.pow_inter_coat_pk.poNum = q.po_det_pk.zzEbeln
2241             AND p.pow_inter_coat_pk.duNum = q.po_det_pk.zzEbelp
2242         WHERE
2243             p.pow_inter_coat_pk.vcode = :vcode
2244             AND p.pow_inter_coat_pk.callRefNum = :call_num
2245             AND p.pow_inter_coat_pk.callType = :callType
2246         """)
2247     public List<Pms_Tpi_Dynamic_Row_Dto> get_dynamic_row_data(String vcode, Integer
call_num, String callType);
2248 }
2249
2250 package com.example.demo.model.bo.insp;
2251
2252 import java.io.Serializable;
2253
2254 import jakarta.persistence.Column;
2255 import jakarta.persistence.Embeddable;
2256 import lombok.AllArgsConstructor;

```

```

2257 import lombok.Builder;
2258 import lombok.Data;
2259 import lombok.NoArgsConstructor;
2260
2261 @NoArgsConstructor
2262 @AllArgsConstructor
2263 @Data
2264 @Builder
2265 @Embeddable
2266 public class Pms_Prime_I_II_PK implements Serializable {
2267
2268     @Column(name = "CALL_REF_NUM", precision = 10)
2269     private Integer callRefNum;
2270
2271     @Column(name = "CALL_TYPE", length = 200)
2272     private String callType;
2273
2274     @Column(name = "VCODE", length = 10)
2275     private String vcode;
2276
2277     @Column(name = "PO_NUM", length = 10)
2278     private String poNum;
2279
2280     @Column(name = "DU_NUM", precision = 5)
2281     private Integer duNum;
2282 }
2283
2284 package com.example.demo.model.vo.insp;
2285
2286 import java.time.LocalDate;
2287 import java.util.List;
2288
2289 import com.fasterxml.jackson.annotation.JsonProperty;
2290
2291 import lombok.AllArgsConstructor;
2292 import lombok.Builder;
2293 import lombok.Data;
2294 import lombok.NoArgsConstructor;
2295
2296 @NoArgsConstructor
2297 @AllArgsConstructor
2298 @Data
2299 @Builder
2300 public class Pms_Prime_I_II_Vo {
2301
2302     /*****/
2303
2304     @JsonProperty("frm_grp_ctrl_vcode")
2305     private String frmGrpCtrlVcode;
2306
2307     @JsonProperty("frm_grp_ctrl_ref")
2308     private Integer frmGrpCtrlRef;
2309
2310     @JsonProperty("frm_grp_ctrl_po")
2311     private String frmGrpCtrlPo;
2312
2313     @JsonProperty("frm_grp_ctrl_call_type")
2314     private String frm_grp_ctrl_call_type;
2315
2316     @JsonProperty("frm_grp_ctrl_paint_type")
2317     private String frm_grp_ctrl_paint_type;
2318
2319     /*****/
2320

```

```
2321 @JsonProperty("paint_spec")
2322 private String paint_spec;
2323
2324 @JsonProperty("paint_shade")
2325 private String paint_shade;
2326
2327 @JsonProperty("paint_manuf")
2328 private String paint_manuf;
2329
2330 @JsonProperty("supplier_trader")
2331 private String supplier_trader;
2332
2333 @JsonProperty("manuf_dt")
2334 private LocalDate manuf_dt;
2335
2336 @JsonProperty("manuf_batch")
2337 private String manuf_batch;
2338
2339 @JsonProperty("manuf_tc")
2340 private String manuf_tc;
2341
2342 @JsonProperty("paint_exp_dt")
2343 private LocalDate paint_exp_dt;
2344
2345 @JsonProperty("mix_ratio")
2346 private String mix_ratio;
2347
2348 @JsonProperty("num_coat")
2349 private String num_coat;
2350
2351 @JsonProperty("time_of_mix")
2352 private String time_of_mix;
2353
2354 @JsonProperty("wet_film_thick")
2355 private String wet_film_thick;
2356
2357 @JsonProperty("spary_gun")
2358 private String spary_gun;
2359
2360 @JsonProperty("hard_dry_time_req")
2361 private String hard_dry_time_req;
2362
2363 @JsonProperty("dft_req")
2364 private String dft_req;
2365
2366 @JsonProperty("dft_act")
2367 private String dft_act;
2368
2369 @JsonProperty("visual_insp")
2370 private String visual_insp;
2371
2372 @JsonProperty("cross_cut_dft_50")
2373 private String cross_cut_dft_50;
2374
2375 @JsonProperty("cross_cut_dft_50_125")
2376 private String cross_cut_dft_50_125;
2377
2378 @JsonProperty("cross_cut_dft_125")
2379 private String cross_cut_dft_125;
2380
2381 /*****/
2382
2383 @JsonProperty("row_form_group")
2384 private List<Pms_Pow_tool_Vo_Row_Group> rowFormGroup;
```

```

2385
2386      /*****/
2387  }
2388
2389  package com.example.demo.model.bo.insp;
2390
2391  import java.math.BigDecimal;
2392  import java.time.LocalDateTime;
2393
2394  import jakarta.persistence.Column;
2395  import jakarta.persistence.EmbeddedId;
2396  import jakarta.persistence.Entity;
2397  import jakarta.persistence.Table;
2398  import lombok.AllArgsConstructor;
2399  import lombok.Builder;
2400  import lombok.Data;
2401  import lombok.NoArgsConstructor;
2402
2403  @NoArgsConstructor
2404  @AllArgsConstructor
2405  @Data
2406  @Builder
2407  @Entity
2408  @Table(name = "PMS_ROOT")
2409  public class Pms_Root {
2410
2411      @EmbeddedId
2412      private Pms_Root_PK pms_root_pk;
2413
2414      @Column(name = "DU_CODE", length = 250)
2415      private String du_code;
2416
2417      @Column(name = "DU_DESC", length = 250)
2418      private String du_desc;
2419
2420      @Column(name = "wbc", length = 24)
2421      private String wbs;
2422
2423      @Column(name = "DU_CALL_QTY", precision = 5)
2424      private Integer duCallQty;
2425
2426      @Column(name = "CALL_TYPE", length = 50)
2427      private String callType;
2428
2429      @Column(name = "CALL_CRT_DT")
2430      private LocalDateTime callCrtDt;
2431
2432      @Column(name = "PAINT_SCHEME", length = 100)
2433      private String paintScheme;
2434
2435      @Column(name = "PAINT_TYPE", length = 100)
2436      private String paintType;
2437
2438      @Column(name = "TC_DOC", length = 50)
2439      private String tcDoc;
2440
2441      @Column(name = "CALL_STAT", length = 50)
2442      private String callStatus;
2443
2444      @Column(name = "ACC_QTY", precision = 10, scale = 2)
2445      private BigDecimal accQty;
2446
2447      @Column(name = "RW_QTY", precision = 10, scale = 2)
2448      private BigDecimal rwQty;

```

```

2449
2450     @Column(name = "REJ_QTY", precision = 10, scale = 2)
2451     private BigDecimal rejQty;
2452
2453     @Column(name = "NR_QTY", precision = 10, scale = 2)
2454     private BigDecimal nrQty;
2455 }
2456
2457 package com.example.demo.dao.insp;
2458
2459 import java.time.LocalDateTime;
2460 import java.util.List;
2461
2462 import org.springframework.data.jpa.repository.JpaRepository;
2463 import org.springframework.data.jpa.repository.Modifying;
2464 import org.springframework.data.jpa.repository.Query;
2465 import org.springframework.data.repository.query.Param;
2466
2467 import com.example.demo.model.bo.insp.Pms_Root;
2468 import com.example.demo.model.bo.insp.Pms_Root_PK;
2469 import com.example.demo.model.dto.insp.Pms_Call_Report_Dto;
2470 import com.example.demo.model.dto.insp.Pms_Root_Dto;
2471 import com.example.demo.model.dto.insp.Pms_Root_Po_Du_Sum_Qty_Dto;
2472
2473 import jakarta.transaction.Transactional;
2474
2475 public interface Pms_Root_dao extends JpaRepository<Pms_Root, Pms_Root_PK> {
2476
2477     /**
2478      * For Given vendor code, get Draft Calls Data
2479      */
2480     @Query("""
2481         SELECT
2482             new com.example.demo.model.dto.insp.Pms_Root_Dto(p)
2483         FROM
2484             Pms_Root p
2485         WHERE
2486             p.pms_root_pk.vcode = :vcode
2487             AND p.callStatus = 'D'
2488         """)
2489     public List<Pms_Root_Dto> get_all_draft_call_data_for_a_vendor(@Param("vcode") String
2490         vcode);
2491
2492     /**
2493      * Get all fresh-Calls for Vendors
2494      */
2495     @Query("""
2496         SELECT
2497             new com.example.demo.model.dto.insp.Pms_Root_Dto(p)
2498         FROM
2499             Pms_Root p
2500         WHERE
2501             p.callStatus='FRESH_BATCH'
2502             AND p.pms_root_pk.vcode = :vcode
2503             AND p.callCrtDt is NULL
2504         """)
2505     public List<Pms_Root_Dto> get_all_fresh_calls(@Param("vcode") String vcode);
2506
2507     /**
2508      * Get all BATCHED-Calls for Feeding Additional Data for the Batch by Vendor
2509      */
2510     @Query("""
2511

```



```

2512         SELECT
2513             new com.example.demo.model.dto.insp.Pms_Root_Dto(p)
2514         FROM
2515             Pms_Root p
2516         WHERE
2517             p.callStatus='MAPPED_BATCH'
2518             AND p.pms_root_pk.vcode = :vcode
2519             AND p.callCrtDt is NOT NULL
2520         """)
2521     public List<Pms_Root_Dto> get_all_batched_calls(@Param("vcode") String vcode);
2522
2523     /**
2524      * Get all Additional-Info-Provided-Calls for approval/rejection by TPI
2525      *
2526      */
2527     @Query("""
2528         SELECT
2529             new com.example.demo.model.dto.insp.Pms_Root_Dto(p)
2530         FROM
2531             Pms_Root p
2532         INNER JOIN
2533             PMS_Batch_Track pt
2534         ON
2535             p.pms_root_pk.vcode = pt.btrack_pk.vcode
2536             AND p.pms_root_pk.callRefNum = pt.btrack_pk.callRefNum
2537             AND p.callType = pt.btrack_pk.callType
2538             AND p.paintType = pt.paintType
2539         WHERE
2540             p.callStatus='ADDI_INFO'
2541             AND p.callCrtDt is NOT NULL
2542             AND sysdate > pt.nextCallTime
2543         ORDER BY
2544             p.pms_root_pk.vcode, p.pms_root_pk.callRefNum,p.pms_root_pk.duNum ASC
2545         """)
2546     public List<Pms_Root_Dto> get_all_addi_calls();
2547
2548     /**
2549      * Find Max CallRefNumber for a given vendor
2550      *
2551      */
2552     @Query("""
2553         SELECT
2554             COALESCE(MAX(p.id.callRefNum), 1000000000)
2555         FROM
2556             Pms_Root p
2557         WHERE
2558             p.id.vcode = :vcode
2559         """)
2560     Integer findMaxCallRefNumByVcode(String vcode);
2561
2562     /**
2563      * Get sum of DU Qty by Combining all Call-Ref/Batch for All PO's
2564      *
2565      */
2566     @Query("""
2567         SELECT
2568             new com.example.demo.model.dto.insp.Pms_Root_Po_Du_Sum_Qty_Dto(
2569                 p.pms_root_pk.poNum,
2570                 p.pms_root_pk.duNum,
2571                 SUM(p.duCallQTY)
2572             )
2573         FROM
2574             Pms_Root p
2575         WHERE
2576             p.pms_root_pk.vcode = :vcode
2577         GROUP BY

```

```

2576         p.pms_root_pk.poNum, p.pms_root_pk.duNum
2577     ORDER BY
2578         p.pms_root_pk.poNum, p.pms_root_pk.duNum
2579     """)
2580 public List<Pms_Root_Po_Du_Sum_Qty_Dto> get_du_sum_qty_for_a_po(@Param("vcode")
String vcode);

2581
2582 /**
2583  * Update Call_Type, Paint_Type, Call_Req_Dt_Tm & TC_Doc_Name for given Batch &
2584  * Vendor Code
2585  */
2586 @Transactional
2587 @Modifying
2588 @Query("""
2589     UPDATE
2590         Pms_Root p
2591     SET
2592         p.callStatus = :batchStatus,
2593         p.callType = :call_type,
2594         p.paintType = :paint_type,
2595         p.tcDoc = :tc_doc,
2596         p.callCrtDt = :call_req_dt_tm
2597     WHERE
2598         p.pms_root_pk.vcode = :vcode
2599         AND p.pms_root_pk.callRefNum = :batch
2600     """)
2601 public int update_root_with_CURRENT_Call_Type_Map(
2602     // BATCH Status !!!
2603     String batchStatus,
2604     // Call Type
2605     String call_type,
2606     // Paint Type
2607     String paint_type,
2608     // TC Document Name
2609     String tc_doc,
2610     // Call Request Date
2611     LocalDateTime call_req_dt_tm,
2612     // Vendor Code
2613     String vcode,
2614     // Batch Number
2615     Integer batch);
2616
2617 /**
2618  * Update CALL_STAT to 'COMP' for given Batch & Vendor Code, once TPI approves
2619  * the call
2620  */
2621 @Transactional
2622 @Modifying
2623 @Query("""
2624     UPDATE
2625         Pms_Root p
2626     SET
2627         p.callStatus = :state
2628     WHERE
2629         p.pms_root_pk.vcode = :vcode
2630         AND p.pms_root_pk.callRefNum = :batch
2631     """)
2632 public int update_root_with_STAGE(
2633     // Call Type
2634     String state,
2635     // Vendor Code
2636     String vcode,
2637     // Batch Number
2638     Integer batch);

```

```

2639
2640 /**
2641  * Report Page for Each Vendors
2642  */
2643 @Query("""
2644     SELECT
2645         new com.example.demo.model.dto.insp.Pms_Call_Report_Dto(a,b,c)
2646     FROM
2647         Pms_Root a
2648     LEFT JOIN
2649         PMS_Batch_Track b
2650         ON a.pms_root_pk.vcode = b.btrack_pk.vcode
2651         AND a.pms_root_pk.callRefNum = b.btrack_pk.callRefNum
2652     LEFT JOIN
2653         Pms_Geo_Img c
2654         ON a.pms_root_pk.vcode = c.pms_geo_pk.vcode
2655         AND a.pms_root_pk.callRefNum =c.pms_geo_pk.callNum
2656         AND a.pms_root_pk.poNum =c.pms_geo_pk.poNum
2657         AND a.pms_root_pk.duNum =c.pms_geo_pk.duNum
2658         AND b.btrack_pk.callType = c.pms_geo_pk.callType
2659     WHERE
2660         a.pms_root_pk.vcode = :vcode
2661     ORDER BY
2662         a.pms_root_pk.vcode, a.pms_root_pk.callRefNum, b.callClrDtTm, a.
2663         pms_root_pk.poNum, a.pms_root_pk.duNum ASC
2664     """)
2665 public List<Pms_Call_Report_Dto> pms_call_report_for_a_vendor(String vcode);
2666
2667 /**
2668  * Report Page for All Vendors
2669  */
2670 /* SELECT
2671     a.vcode,
2672     a.CALL_REF_NUM,
2673     a.PO_NUM,
2674     a.WBC,
2675     a.DU_NUM,
2676     a.DU_CODE,
2677     a.DU_DESC,
2678     a.DU_CALL_QTY,
2679     a.PAINT_SCHEME,
2680
2681     b.CALL_TYPE,
2682     b.PAINT_TYPE,
2683     CASE
2684         WHEN CALL_STATUS IS NOT NULL THEN
2685             b.CALL_STATUS
2686         ELSE
2687             a.CALL_STAT
2688     END,
2689     b.CALL_REQ_DT_TM,
2690     b.CALL_CLR_DT_TM,
2691     b.NEXT_CALL_TIME,
2692     b.ACT_BY,
2693
2694     c.GEO_LONG,
2695     c.GEO_LATI,
2696     c.IMG_NAME
2697 FROM
2698     Pms_Root a
2699 LEFT JOIN
2700     PMS_Batch_Track b
2701     ON a.vcode = b.V_CODE
2702     AND a.CALL_REF_NUM = b.CALL_REF_NUL

```

```

2702 LEFT JOIN
2703     Pms_Geo_Img c
2704     ON a.vcode = c.VCODE
2705     AND a.CALL_REF_NUM =c.CALL_NUM
2706     AND a.PO_NUM =c.PO_NUM
2707     AND a.DU_NUM =c.DU_NUM
2708     AND b.CALL_TYPE = c.CALL_TYPE
2709 WHERE
2710     (
2711         (
2712             CALL_STATUS IS NOT NULL
2713             AND b.CALL_STATUS = 'REJ'
2714         )
2715         OR
2716         (
2717             CALL_STATUS IS NULL
2718             AND a.CALL_STAT = 'REJ'
2719         )
2720     )
2721 AND
2722     b.CALL_TYPE = 'Blasting to Sa 2.5'
2723 AND CALL_REQ_DT_TM between to_date('14-11-25', 'dd-mm-yy') AND
    to_date('15-11-25', 'dd-mm-yy')
2724 ORDER BY
2725     a.vcode, a.CALL_REF_NUM, b.CALL_REQ_DT_TM, a.PO_NUM, a.DU_NUM ASC;
2726 */
2727 @Query("""
2728     SELECT
2729         new com.example.demo.model.dto.insp.Pms_Call_Report_Dto(a,b,c)
2730     FROM
2731         Pms_Root a
2732     LEFT JOIN
2733         PMS_Batch_Track b
2734         ON a.pms_root_pk.vcode = b.btrack_pk.vcode
2735         AND a.pms_root_pk.callRefNum = b.btrack_pk.callRefNum
2736     LEFT JOIN
2737         Pms_Geo_Img c
2738         ON a.pms_root_pk.vcode = c.pms_geo_pk.vcode
2739         AND a.pms_root_pk.callRefNum =c.pms_geo_pk.callNum
2740         AND a.pms_root_pk.poNum =c.pms_geo_pk.poNum
2741         AND a.pms_root_pk.duNum =c.pms_geo_pk.duNum
2742         AND b.btrack_pk.callType = c.pms_geo_pk.callType
2743     ORDER BY
2744         a.pms_root_pk.vcode, a.pms_root_pk.callRefNum, b.callClrDtTm, a.
        pms_root_pk.poNum, a.pms_root_pk.duNum ASC
2745     """)
2746 public List<Pms_Call_Report_Dto> pms_call_report_all_vendor();
2747
2748 /**
2749  * Get Call/Batch Details for given Vendor-Code and Call-Number
2750  */
2751 @Query("""
2752     SELECT
2753         new com.example.demo.model.dto.insp.Pms_Root_Dto(p)
2754     FROM
2755         Pms_Root p
2756     WHERE
2757         p.pms_root_pk.vcode = :vcode
2758         AND p.pms_root_pk.callRefNum = :batch
2759     ORDER BY
2760         p.pms_root_pk.vcode,
2761         p.pms_root_pk.callRefNum ASC
2762     """)
2763 public List<Pms_Root_Dto> get_batch_details(@Param("vcode") String vcode, @Param(

```

```

2764     "batch") Integer callNum);
2765 }
2766
2767 package com.example.demo.model.dto.insp;
2768
2769 import java.math.BigDecimal;
2770 import java.time.LocalDateTime;
2771 import java.util.Date;
2772
2773 import com.example.demo.model.bo.insp.Pms_Root;
2774 import com.fasterxml.jackson.annotation.JsonFormat;
2775
2776 import lombok.AllArgsConstructor;
2777 import lombok.Data;
2778 import lombok.NoArgsConstructor;
2779
2780 @NoArgsConstructor
2781 @AllArgsConstructor
2782 @Data
2783 public class Pms_Root_Dto {
2784
2785     private Integer callRefNum;
2786     private String vcode;
2787     private String poNum;
2788     private Integer duNum;
2789
2790     private String du_desc;
2791     private String wbs;
2792     private Integer duCallQTY;
2793     private String callType;
2794
2795     @JsonFormat(shape = JsonFormat.Shape.STRING, pattern = "yyyy-MM-dd HH:mm:ss",
2796         timezone = "Asia/Kolkata")
2797     private LocalDateTime callCrtDt;
2798     private String paintScheme;
2799     private String paintType;
2800     private String tcDoc;
2801     private String callStatus;
2802
2803     private BigDecimal accQty;
2804     private BigDecimal rwQty;
2805     private BigDecimal rejQty;
2806     private BigDecimal nrQty;
2807
2808     /**
2809      * Custom Constructor for Custom_Query in DAO
2810      *
2811      * @param entity
2812      */
2813     public Pms_Root_Dto(Pms_Root entity) {
2814
2815         /**
2816          * For Pms_Root_PK class
2817          */
2818         if (entity != null && entity.getPms_root_pk() != null) {
2819             this.callRefNum = entity.getPms_root_pk().getCallRefNum();
2820             this.vcode = entity.getPms_root_pk().getVcode();
2821             this.poNum = entity.getPms_root_pk().getPoNum();
2822             this.duNum = entity.getPms_root_pk().getDuNum();
2823         }
2824
2825         /**
2826          * For Pms_Root class

```

```

2826         */
2827         this.du_desc = entity.getDu_desc();
2828         this.wbs = entity.getWbs();
2829         this.duCallQTY = entity.getDuCallQTY();
2830         this.callType = entity.getCallType();
2831         this.callCrtDt = entity.getCallCrtDt();
2832         this.paintScheme = entity.getPaintScheme();
2833         this.paintType = entity.getPaintType();
2834         this.tcDoc = entity.getTcDoc();
2835         this.callStatus = entity.getCallStatus();
2836
2837         this.accQty = entity.getAccQty();
2838         this.rwQty = entity.getRwQty();
2839         this.rejQty = entity.getRejQty();
2840         this.nrQty = entity.getNrQty();
2841     }
2842 }
2843
2844 package com.example.demo.model.bo.insp;
2845
2846 import java.io.Serializable;
2847
2848 import jakarta.persistence.Column;
2849 import jakarta.persistence.Embeddable;
2850 import lombok.AllArgsConstructor;
2851 import lombok.Builder;
2852 import lombok.Data;
2853 import lombok.NoArgsConstructor;
2854
2855 @NoArgsConstructor
2856 @AllArgsConstructor
2857 @Data
2858 @Builder
2859 @Embeddable
2860 public class Pms_Root_PK implements Serializable {
2861
2862     @Column(name = "CALL_REF_NUM", precision = 10)
2863     private Integer callRefNum;
2864
2865     @Column(name = "VCODE", length = 10)
2866     private String vcode;
2867
2868     @Column(name = "PO_NUM", length = 10)
2869     private String poNum;
2870
2871     @Column(name = "DU_NUM", precision = 5)
2872     private Integer duNum;
2873
2874 }
2875
2876 package com.example.demo.model.dto.insp;
2877
2878 import lombok.Builder;
2879 import lombok.Data;
2880 import lombok.NoArgsConstructor;
2881
2882 @NoArgsConstructor
2883 @Data
2884 @Builder
2885 public class Pms_Root_Po_Du_Sum_Qty_Dto {
2886
2887     private String poNum;
2888     private Integer duNum;
2889     private Long duSumCallQTY;

```

```

2890
2891     public Pms_Root_Po_Du_Sum_Qty_Dto(String poNum, Integer duNum, Long duSumCallQTY) {
2892
2893         this.poNum = poNum;
2894         this.duNum = duNum;
2895         this.duSumCallQTY = duSumCallQTY;
2896
2897     }
2898
2899 }
2900
2901 package com.example.demo.service.insp;
2902
2903 import java.io.IOException;
2904 import java.nio.file.Files;
2905 import java.nio.file.Path;
2906 import java.nio.file.Paths;
2907 import java.nio.file.StandardCopyOption;
2908 import java.time.LocalDate;
2909 import java.time.LocalDateTime;
2910 import java.time.format.DateTimeFormatter;
2911 import java.util.ArrayList;
2912 import java.util.List;
2913 import java.util.Optional;
2914
2915 import org.springframework.beans.factory.annotation.Value;
2916 import org.springframework.security.core.Authentication;
2917 import org.springframework.stereotype.Service;
2918 import org.springframework.transaction.annotation.Transactional;
2919 import org.springframework.web.multipart.MultipartFile;
2920
2921 import com.example.demo.dao.insp.PMS_Batch_Track_Dao;
2922 import com.example.demo.dao.insp.PMS_Paint_Curing_Time_Dao;
2923 import com.example.demo.dao.insp.PMS_Po_Details_Dao;
2924 import com.example.demo.dao.insp.Pms_Blast_2_5_Dao;
2925 import com.example.demo.dao.insp.Pms_Fnl_Qty_Dao;
2926 import com.example.demo.dao.insp.Pms_Geo_Img_Dao;
2927 import com.example.demo.dao.insp.Pms_Pow_Tool_Dao;
2928 import com.example.demo.dao.insp.Pms_Prime_I_II_Dao;
2929 import com.example.demo.dao.insp.Pms_Root_dao;
2930 import com.example.demo.model.bo.insp.PMS_Batch_Track;
2931 import com.example.demo.model.bo.insp.PMS_Batch_Track_PK;
2932 import com.example.demo.model.bo.insp.PMS_Paint_Curing_Time;
2933 import com.example.demo.model.bo.insp.Pms_Blast_2_5;
2934 import com.example.demo.model.bo.insp.Pms_Blast_2_5_PK;
2935 import com.example.demo.model.bo.insp.Pms_Fnl_Qty;
2936 import com.example.demo.model.bo.insp.Pms_Fnl_Qty_PK;
2937 import com.example.demo.model.bo.insp.Pms_Pow_Tool;
2938 import com.example.demo.model.bo.insp.Pms_Pow_Tool_PK;
2939 import com.example.demo.model.bo.insp.Pms_Prime_I_II;
2940 import com.example.demo.model.bo.insp.Pms_Prime_I_II_PK;
2941 import com.example.demo.model.bo.insp.Pms_Root;
2942 import com.example.demo.model.bo.insp.Pms_Root_PK;
2943 import com.example.demo.model.dto.insp.PMS_Batch_Track_Dto;
2944 import com.example.demo.model.dto.insp.Pms_Call_Report_Dto;
2945 import com.example.demo.model.dto.insp.Pms_Du_Img_GLoc_Dto;
2946 import com.example.demo.model.dto.insp.Pms_Fnl_Qty_Dto;
2947 import com.example.demo.model.dto.insp.Pms_Po_Details_Dto;
2948 import com.example.demo.model.dto.insp.Pms_Root_Dto;
2949 import com.example.demo.model.dto.insp.Pms_Root_Po_Du_Sum_Qty_Dto;
2950 import com.example.demo.model.dto.insp.Pms_Tpi_Dynamic_Row_Dto;
2951 import com.example.demo.model.vo.insp.Pms_Blast_sa_2_5_Vo;
2952 import com.example.demo.model.vo.insp.Pms_Call_Vo_row_form_Array;
2953 import com.example.demo.model.vo.insp.Pms_Pow_tool_Vo;

```

```

2954 import com.example.demo.model.vo.insp.Pms_Prime_I_II_Vo;
2955 import com.fasterxml.jackson.databind.ObjectMapper;
2956
2957 @Service
2958 public class PMS_Serv {
2959
2960     /**
2961      * File Save Path
2962      */
2963     @Value("${file.upload-dir}")
2964     private String UPLOAD_DIR;
2965
2966     private final Pms_Root_dao pms_root_dao;
2967     private final Pms_Pow_Tool_Dao pms_ptool_dao;
2968     private final Pms_Blast_2_5_Dao pms_blast_dao;
2969     private final Pms_Prime_I_II_Dao pms_inter_coat_dao;
2970     private final Pms_Fnl_Qty_Dao pms_fnl_qty_dao;
2971     private final PMS_Batch_Track_Dao pms_btrack_dao;
2972     private final PMS_Paint_Curing_Time_Dao paint_cur_dao;
2973     private final Pms_Geo_Img_Dao pms_geo;
2974     private final PMS_Po_Details_Dao pms_po_det_dao;
2975
2976     public PMS_Serv(Pms_Root_dao pms_root_dao, Pms_Pow_Tool_Dao pms_ptool_dao,
2977         Pms_Blast_2_5_Dao pms_blast_dao,
2978         Pms_Prime_I_II_Dao pms_inter_coat_dao, Pms_Fnl_Qty_Dao pms_fnl_qty_dao,
2979         PMS_Batch_Track_Dao pms_btrack_dao,
2980         PMS_Paint_Curing_Time_Dao paint_cur_dao, Pms_Geo_Img_Dao pms_geo,
2981         PMS_Po_Details_Dao pms_po_det_dao) {
2982
2983         this.pms_root_dao = pms_root_dao;
2984         this.pms_ptool_dao = pms_ptool_dao;
2985         this.pms_blast_dao = pms_blast_dao;
2986         this.pms_inter_coat_dao = pms_inter_coat_dao;
2987         this.pms_fnl_qty_dao = pms_fnl_qty_dao;
2988         this.pms_btrack_dao = pms_btrack_dao;
2989         this.paint_cur_dao = paint_cur_dao;
2990         this.pms_geo = pms_geo;
2991         this.pms_po_det_dao = pms_po_det_dao;
2992     }
2993
2994     /**
2995      * Get Pending Draft Calls for a vendor
2996      */
2997     public List<Pms_Root_Dto> get_all_draft_calls_for_a_vendor(String vcode) {
2998
2999         return pms_root_dao.get_all_draft_call_data_for_a_vendor(vcode);
3000     }
3001
3002     /**
3003      * Returns PO Details for a specific Vendor code
3004      *
3005      * @param vcode
3006      * @return
3007      */
3008     public List<Pms_Po_Details_Dto> get_all_po_by_vendor(String vcode) {
3009
3010         return pms_po_det_dao.get_po_det_by_vendor(vcode);
3011     }
3012
3013     /**
3014      * Returns PO Details as first & sb-vendor
3015      */

```



```

3015     * @param vcode
3016     * @return
3017     */
3018     public List<Pms_Po_Details_Dto> get_po_for_2_nd_vendor(String vcode) {
3019
3020         return pms_po_det_dao.get_po_for_2_nd_vendor(vcode);
3021     }
3022
3023     /**
3024     * Get all FERSH-Calls for BATCHING-Process
3025     */
3026     public List<Pms_Root_Dto> get_all_fresh_calls(String vcode) {
3027
3028         return pms_root_dao.get_all_fresh_calls(vcode);
3029     }
3030
3031
3032     /**
3033     * Get all BATCHED-Calls for Feeding Additional Data for the Batch by Vendor
3034     */
3035     public List<Pms_Root_Dto> get_all_batched_calls(String vcode) {
3036
3037         return pms_root_dao.get_all_batched_calls(vcode);
3038     }
3039
3040
3041     /**
3042     * Get all Additional-Info-Provided-Calls for approval/rejection by TPI
3043     */
3044     public List<Pms_Root_Dto> get_all_addi_calls() {
3045
3046         return pms_root_dao.get_all_addi_calls();
3047     }
3048
3049
3050     /*****
3051
3052     */
3053     *
3054     */
3055     public List<Pms_Tpi_Dynamic_Row_Dto> get_row_data(String vcode, String batch, String
call_type) {
3056
3057         if (call_type.equals("Power Tool cleaning")) {
3058
3059             return pms_ptool_dao.get_dynamic_row_data(vcode, Integer.parseInt(batch));
3060         }
3061
3062         if (call_type.equals("Blasting to Sa 2.5")) {
3063
3064             return pms_blast_dao.get_dynamic_row_data(vcode, Integer.parseInt(batch));
3065         }
3066
3067         if (!call_type.equals("Power Tool cleaning") && !call_type.equals("Blasting to
Sa 2.5")) {
3068
3069             return pms_inter_coat_dao.get_dynamic_row_data(vcode, Integer.parseInt(batch
), call_type);
3070         }
3071
3072         return null;
3073
3074
3075

```

```

3076     }
3077
3078     /**
3079     * Get sum of DU Qty by Combining all Call-Ref/Batch for All PO's
3080     */
3081     public List<Pms_Root_Po_Du_Sum_Qty_Dto> get_du_sum_qty_for_a_po(String vCode) {
3082
3083         return pms_root_dao.get_du_sum_qty_for_a_po(vCode);
3084     }
3085
3086     /**
3087     * Save New PMS Call Request
3088     */
3089     @Transactional(rollbackFor = Exception.class)
3090     public boolean save_pms_call(String po_no, String paint_schm_ref, String project,
3091         String wo_no, String duCall,
3092         String vcode) throws Exception {
3093
3094         /**
3095         * De-serialize duCall JSON-String into List<DuCall>
3096         */
3097         ObjectMapper objectMap_from_JSON_String = new ObjectMapper();
3098
3099         List<Pms_Call_Vo_row_form_Array> row_formArray = new ArrayList<>();
3100         try {
3101
3102             row_formArray = objectMap_from_JSON_String.readValue(duCall,
3103                 objectMap_from_JSON_String.getTypeFactory()
3104                     .constructCollectionType(List.class, Pms_Call_Vo_row_form_Array.class
3105                     ));
3106
3107         } catch (Exception e) {
3108             System.out.println("Error");
3109         }
3110
3111         Integer next_call_num = pms_root_dao.findMaxCallRefNumByVcode(vcode) + 1;
3112
3113         /**
3114         * Iterate for Each Row from the request
3115         */
3116         row_formArray.forEach((one_data) -> {
3117
3118             Pms_Root_PK pk = Pms_Root_PK.builder()
3119                 // Call Number
3120                 .callRefNum(next_call_num)
3121                 // Vendor Code
3122                 .vcode(vcode)
3123                 // PO Number
3124                 .poNum(po_no)
3125                 // DU Number
3126                 .duNum(Integer.parseInt(one_data.getDu()))
3127                 // build!
3128                 .build();
3129
3130             Pms_Root data = Pms_Root.builder()
3131                 // Primary Key
3132                 .pms_root_pk(pk)
3133                 // DU_Code
3134                 .du_code(one_data.getDu_code())
3135                 // DU_Desc
3136                 .du_desc(one_data.getDesc())
3137                 // WBS
3138                 .wbs(one_data.getDu_wbs())
3139                 // Call Request Quantity

```

```

3137         .duCallQTY(one_data.getQty())
3138     // Call Type
3139     // .callType(call_type)
3140     // Call Request Data
3141     // .callCrtDt(LocalDateTime.parse(call_dt_tm))
3142     // Paint Scheme
3143     .paintScheme(paint_schm_ref)
3144     // Paint Type
3145     // .paintType(paint_type)
3146     // Paint TC Document Name
3147     // .tcDoc("tc.pdf")
3148     // Call Status
3149     .callStatus("FRESH_BATCH")
3150     // build!
3151     .build();
3152
3153     var saved_data = pms_root_dao.save(data);
3154
3155     /* Record in FNL_QTY Table for Tracking */
3156
3157     Pms_Fnl_Qty_PK pk_fnl = Pms_Fnl_Qty_PK.builder()
3158         // Vendor Code
3159         .vcode(vcode)
3160         // PO
3161         .po(po_no)
3162         // DU_NUM
3163         .poSno(Integer.parseInt(one_data.getDu()))
3164         // Build!
3165         .build();
3166
3167     //////////////////////////////////////
3168
3169     Double existing_qty = 0.0;
3170
3171     Optional<Pms_Fnl_Qty> single_du_record_optional = pms_fnl_qty_dao.findById(
3172         pk_fnl);
3173
3174     if (single_du_record_optional.isPresent()) {
3175         existing_qty = single_du_record_optional.get().getAccQty();
3176     }
3177
3178     Double cumulative_qty = one_data.getQty() + existing_qty;
3179
3180     //////////////////////////////////////
3181
3182     String currentYear = String.valueOf(LocalDate.now().getYear());
3183
3184     Pms_Fnl_Qty fnl = Pms_Fnl_Qty.builder()
3185         // PK
3186         .pms_fnl_pk(pk_fnl)
3187         // Accepted Quantity
3188         .accQty(cumulative_qty)
3189         // Accepted Year
3190         .pmsCallYr(currentYear)
3191         // Build!
3192         .build();
3193
3194     pms_fnl_qty_dao.save(fnl);
3195
3196     });
3197
3198     return true;
3199

```

```

3200     }
3201
3202     /**
3203     * Record Additional Information for a 'Power Tool Cleaning'
3204     */
3205     @Transactional(rollbackFor = Exception.class)
3206     public boolean record_addi_info_ptc(Pms_Pow_tool_Vo form_input, Authentication
3207     authenticated_obj) throws Exception {
3208
3209         form_input.getRowFormGroup().forEach((row_data) -> {
3210
3211             Pms_Pow_Tool_PK pk = Pms_Pow_Tool_PK.builder()
3212                 // Call Request Number
3213                 .callRefNum(form_input.getFrmGrpCtrlRef())
3214                 // Call's Vendor Code
3215                 .vcode(form_input.getFrmGrpCtrlVcode())
3216                 // Call's PO
3217                 .poNum(form_input.getFrmGrpCtrlPo())
3218                 // Call's DU - With Iteration
3219                 .duNum(row_data.getDu())
3220                 // Build !
3221                 .build();
3222
3223             Pms_Pow_Tool record = Pms_Pow_Tool.builder()
3224                 // Primary Key
3225                 .pow_tool_pk(pk)
3226                 // DU Description
3227                 .duDesc(row_data.getDuDescription())
3228                 // DU Call Quantity
3229                 .duCallQty(row_data.getQuantity())
3230                 // Dry Bulb Temperature
3231                 .dryBulbT(row_data.getDryBulbTemp())
3232                 // Wet Bulb Temperature
3233                 .wetBulbT(row_data.getWetBulbTemp())
3234                 // Dew Point
3235                 .dewPt(row_data.getDewPoint())
3236                 // RH
3237                 .rh(row_data.getRh())
3238                 // Reading Date
3239                 .readDt(row_data.getReading_date())
3240                 // Start Time
3241                 .readTimeSt(row_data.getStartTime())
3242                 // End Time
3243                 .readTimeEnd(row_data.getEndTime())
3244                 // Remark
3245                 .inspRmrk(row_data.getRemark())
3246                 // Cleaned By
3247                 .cleanBy(form_input.getClean_used())
3248                 // Rust Grade
3249                 .rawRustGrade(form_input.getRaw_matl_rust_grade())
3250                 // Dust Free Check Result
3251                 .dust_free_check_result(form_input.getDust_free_check_result())
3252                 // Visual Inspection
3253                 .visInsp(form_input.getVisual_insp())
3254                 // Cleared By
3255                 .inspClrBy(authenticated_obj.getName())
3256                 // Cleared On
3257                 .inspClrDt(LocalDate.now())
3258                 // Build !
3259                 .build();
3260
3261             pms_ptool_dao.save(record);
3262
3263         });

```

```

3263
3264 /**
3265  * Update Batch-Track Table Too to mark this call as completed only
3266  */
3267 LocalDateTime currentTime = LocalDateTime.now();
3268 LocalDateTime nextCallTime = null;
3269 Integer callRefNum = form_input.getFrmGrpCtrlRef();
3270 String vcode = form_input.getFrmGrpCtrlVcode();
3271
3272 LocalDate today = LocalDate.now();
3273 int fyYear;
3274
3275 if (today.getMonthValue() < 4) { // Jan, Feb, Mar
3276     fyYear = today.getYear() - 1;
3277 } else { // Apr to Dec
3278     fyYear = today.getYear();
3279 }
3280
3281 String inspFyYear = String.valueOf(fyYear);
3282
3283 String callType = form_input.getFrm_grp_ctrl_call_type();
3284
3285 String paint_type = form_input.getFrm_grp_ctrl_paint_type();
3286 PMS_Paint_Curing_Time paint_curing_data = paint_cur_dao.findById(paint_type).get
3287 ();
3288 nextCallTime = currentTime.plusHours(paint_curing_data.getCuring_time().longValue
3289 ());
3290
3291 pms_btrack_dao.update_next_call_time(currentTime, nextCallTime, callRefNum, vcode
3292 , inspFyYear, callType);
3293
3294 /**
3295  * Reset PMS_ROOT table's CALL_STAT to 'ADDI_INFO' to indicate additional Inputs
3296  * are given for the call-type
3297  */
3298 pms_root_dao.update_root_with_STAGE("ADDI_INFO", authed_obj.getName(), form_input
3299 .getFrmGrpCtrlRef());
3300
3301 /**
3302  * Update in Track-Table too ::: WIP
3303  */
3304
3305 return true;
3306
3307 }
3308
3309 /**
3310  * Record Additional Information for 'Blast-SA'
3311  */
3312 @Transactional(rollbackFor = Exception.class)
3313 public boolean record_addi_blast(Pms_Blast_sa_2_5_Vo form_input, Authentication
3314 authed_obj) throws Exception {
3315
3316     form_input.getRowFormGroup().forEach((row_data) -> {
3317
3318         Pms_Blast_2_5_PK pk = Pms_Blast_2_5_PK.builder()
3319             // Call Request Number
3320             .callRefNum(form_input.getFrmGrpCtrlRef())
3321             // Call's Vendor Code
3322             .vcode(form_input.getFrmGrpCtrlVcode())
3323             // Call's PO
3324             .poNum(form_input.getFrmGrpCtrlPo())
3325             // Call's DU - With Iteration
3326             .duNum(row_data.getDu())

```

```

3322         // Build !
3323         .build();
3324
3325     Pms_Blast_2_5 recrod = Pms_Blast_2_5.builder()
3326         /*****
3327         // Primary Key
3328         .blast_2_5_pk(pk)
3329         // DU Description
3330         .duDesc(row_data.getDuDescription())
3331         // DU Call Quantity
3332         .duCallQty(row_data.getQuantity())
3333         // Dry Bulb Temperature
3334         .dryBulbT(row_data.getDryBulbTemp())
3335         // Wet Bulb Temperature
3336         .wetBulbT(row_data.getWetBulbTemp())
3337         // Dew Point
3338         .dewPt(row_data.getDewPoint())
3339         // RH
3340         .rh(row_data.getRh())
3341         // Reading Date
3342         .readDt(row_data.getReading_date())
3343         // Start Time
3344         .readTimeSt(row_data.getStartTime())
3345         // End Time
3346         .readTimeEnd(row_data.getEndTime())
3347         // Remark
3348         .inspRmrk(row_data.getRemark())
3349         *****/
3350         // Blasting Medium
3351         .blast_medium(form_input.getBlast_medium())
3352         // Rust Grade
3353         .raw_matl_rust_grade(form_input.getRaw_matl_rust_grade())
3354         // Moist Content Test
3355         .moist_cont(form_input.getMoist_cont())
3356         // Surface Chloride
3357         .surf_chloride(form_input.getSurf_chloride())
3358         // Profile Gauge Reading
3359         .surf_prof_gauge_read(form_input.getSurf_prof_gauge_read())
3360         // Checked Date & Time
3361         .checked_dt_tm(form_input.getChecked_dt_tm())
3362         // Dust Free Check Result
3363         .dust_free_check(form_input.getDust_free_check())
3364         // Visual Inspection
3365         .visual_insp(form_input.getVisual_insp())
3366         /*****
3367         // Cleared By
3368         .inspClrBy(authed_obj.getName())
3369         // Cleared On
3370         .inspClrDt(LocalDate.now())
3371         *****/
3372         // Build !
3373         .build();
3374
3375     var saved_data = pms_blast_dao.save(recrod);
3376
3377 });
3378
3379 /**
3380  * Update Batch-Track Table Too to mark this call as completed only
3381  */
3382 LocalDateTime currentTime = LocalDateTime.now();
3383 LocalDateTime nextCallTime = null;
3384 Integer callRefNum = form_input.getFrmGrpCtrlRef();
3385 String vcode = form_input.getFrmGrpCtrlVcode();

```

```

3386
3387     LocalDate today = LocalDate.now();
3388     int fyYear;
3389
3390     if (today.getMonthValue() < 4) { // Jan, Feb, Mar
3391         fyYear = today.getYear() - 1;
3392     } else { // Apr to Dec
3393         fyYear = today.getYear();
3394     }
3395
3396     String inspFyYear = String.valueOf(fyYear);
3397
3398     String callType = form_input.getFrm_grp_ctrl_call_type();
3399
3400     String paint_type = form_input.getFrm_grp_ctrl_paint_type();
3401     PMS_Paint_Curing_Time paint_curing_data = paint_cur_dao.findById(paint_type).get
3402     ();
3403     nextCallTime = currentTime.plusHours(paint_curing_data.getCuring_time().longValue
3404     ());
3405
3406     pms_btrack_dao.update_next_call_time(currentTime, nextCallTime, callRefNum, vcode
3407     , inspFyYear, callType);
3408
3409     /**
3410      * Reset PMS_ROOT table's CALL_STAT to 'ADDI_INFO' to indicate additional Inputs
3411      * are given for the call-type
3412      */
3413     pms_root_dao.update_root_with_STAGE("ADDI_INFO", authed_obj.getName(), form_input
3414     .getFrmGrpCtrlRef());
3415
3416     /**
3417      * Update in Track-Table too ::: WIP
3418      */
3419
3420     return true;
3421 }
3422
3423 /**
3424  * Record Additional Information for 'Prime-coat' and 'Others' call-types
3425  */
3426 @Transactional(rollbackFor = Exception.class)
3427 public boolean record_addi_prime_others(Pms_Prime_I_II_Vo form_input, Authentication
3428 authed_obj) throws Exception {
3429
3430     form_input.getRowFormGroup().forEach((row_data) -> {
3431
3432         Pms_Prime_I_II_PK pk = Pms_Prime_I_II_PK.builder()
3433             // Call Request Number
3434             .callRefNum(form_input.getFrmGrpCtrlRef())
3435             // Call Type
3436             .callType(form_input.getFrm_grp_ctrl_call_type())
3437             // Call's Vendor Code
3438             .vcode(form_input.getFrmGrpCtrlVcode())
3439             // Call's PO
3440             .poNum(form_input.getFrmGrpCtrlPo())
3441             // Call's DU - With Iteration
3442             .duNum(row_data.getDu())
3443             // Build !
3444             .build();
3445
3446         /*****
3447
3448         Pms_Prime_I_II recrod = Pms_Prime_I_II.builder()
3449             /*****

```

```
3445 // Primary Key
3446 .pow_inter_coat_pk(pk)
3447 // DU Description
3448 .duDesc(row_data.getDuDescription())
3449 // DU Call Quantity
3450 .duCallQty(row_data.getQuantity())
3451 // Dry Bulb Temperature
3452 .dryBulbT(row_data.getDryBulbTemp())
3453 // Wet Bulb Temperature
3454 .wetBulbT(row_data.getWetBulbTemp())
3455 // Metal Surface Temperature
3456 .matelSurfT(row_data.getExtra())
3457 // Dew Point
3458 .dewPt(row_data.getDewPoint())
3459 // RH
3460 .rh(row_data.getRh())
3461 // Reading Date
3462 .readDt(row_data.getReading_date())
3463 // Start Time
3464 .readTimeSt(row_data.getStartTime())
3465 // End Time
3466 .readTimeEnd(row_data.getEndTime())
3467 // Remark
3468 .inspRmrk(row_data.getRemark())
3469 /*****/
3470 // Paint Specification
3471 .paint_spec(form_input.getPaint_spec())
3472 // paint Shade
3473 .paint_shade(form_input.getPaint_shade())
3474 // Paint Manufacturer
3475 .paint_manuf(form_input.getPaint_manuf())
3476 // Supplier/ Trader
3477 .supplier_trader(form_input.getSupplier_trader())
3478 // Paint Manufacturing Date
3479 .manuf_dt(form_input.getManuf_dt())
3480 // Manufacturing Batch
3481 .manuf_batch(form_input.getManuf_batch())
3482 // Paint TC
3483 .manuf_tc(form_input.getManuf_tc())
3484 // Paint Expire Date
3485 .paint_exp_dt(form_input.getPaint_exp_dt())
3486 // Mix-Ratio
3487 .mix_ratio(form_input.getMix_ratio())
3488 // Number of Coat
3489 .num_coat(form_input.getNum_coat())
3490 // Duration of Mix
3491 .time_of_mix(form_input.getTime_of_mix())
3492 // Wet Film Thickness
3493 .wet_film_thick(form_input.getWet_film_thick())
3494 // Spray Gun
3495 .spary_gun(form_input.getSpary_gun())
3496 // Hard Dry Time Required
3497 .hard_dry_time_req(form_input.getHard_dry_time_req())
3498 // DFT Required
3499 .dft_req(form_input.getDft_req())
3500 // DTF Actual Measurement
3501 .dft_act(form_input.getDft_act())
3502 // Visual Inspection
3503 .visual_insp(form_input.getVisual_insp())
3504 // Cross Cut Test upto 50u
3505 .cross_cut_dft_50(form_input.getCross_cut_dft_50())
3506 // Cross Cut Test 50 - 125u
3507 .cross_cut_dft_50_125(form_input.getCross_cut_dft_50_125())
3508 // Cross Cut Test above 125u
```



```

3509         .cross_cut_dft_125(form_input.getCross_cut_dft_125())
3510         /*****
3511         // Cleared By
3512         .inspClrBy(authed_obj.getName())
3513         // Cleared On
3514         .inspClrDt(LocalDate.now())
3515         *****/
3516         // Build !
3517         .build();
3518
3519         var saved_data = pms_inter_coat_dao.save(recrod);
3520
3521     });
3522
3523     /**
3524     * Update Batch-Track Table Too to mark this call as completed only
3525     */
3526     LocalDateTime currentTime = LocalDateTime.now();
3527     LocalDateTime nextCallTime = null;
3528     Integer callRefNum = form_input.getFrmGrpCtrlRef();
3529     String vcode = form_input.getFrmGrpCtrlVcode();
3530
3531     LocalDate today = LocalDate.now();
3532     int fyYear;
3533
3534     if (today.getMonthValue() < 4) { // Jan, Feb, Mar
3535         fyYear = today.getYear() - 1;
3536     } else { // Apr to Dec
3537         fyYear = today.getYear();
3538     }
3539
3540     String inspFyYear = String.valueOf(fyYear);
3541
3542     String callType = form_input.getFrm_grp_ctrl_call_type();
3543
3544     String paint_type = form_input.getFrm_grp_ctrl_paint_type();
3545     PMS_Paint_Curing_Time paint_curing_data = paint_cur_dao.findById(paint_type).get
3546     ();
3547     nextCallTime = currentTime.plusHours(paint_curing_data.getCuring_time().longValue
3548     ());
3549
3550     pms_btrack_dao.update_next_call_time(currentTime, nextCallTime, callRefNum, vcode
3551     , inspFyYear, callType);
3552
3553     /**
3554     * Reset PMS_ROOT table's CALL_STAT to 'ADDI_INFO' to indicate additional Inputs
3555     * are given for the call-type
3556     */
3557     pms_root_dao.update_root_with_STAGE("ADDI_INFO", authed_obj.getName(), form_input
3558     .getFrmGrpCtrlRef());
3559
3560     return true;
3561 }
3562
3563 /**
3564 * Record Power-Tool-Cleaning Call-Data
3565 */
3566 @Transactional(rollbackFor = Exception.class)
3567 public boolean record_power_tool_cleaning_call(Pms_Pow_tool_Vo form_input,
3568 Authentication authed_obj, String status)
3569     throws Exception {
3570
3571     form_input.getRowFormGroup().forEach((row_data) -> {

```

```

3568 Pms_Pow_Tool_PK pk = Pms_Pow_Tool_PK.builder()
3569     // Call Request Number
3570     .callRefNum(form_input.getFrmGrpCtrlRef())
3571     // Call's Vendor Code
3572     .vcode(form_input.getFrmGrpCtrlVcode())
3573     // Call's PO
3574     .poNum(form_input.getFrmGrpCtrlPo())
3575     // Call's DU - With Iteration
3576     .duNum(row_data.getDu())
3577     // Build !
3578     .build();
3579
3580 Pms_Pow_Tool record = Pms_Pow_Tool.builder()
3581     // Primary Key
3582     .pow_tool_pk(pk)
3583     // DU Description
3584     .duDesc(row_data.getDuDescription())
3585     // DU Call Quantity
3586     .duCallQty(row_data.getQuantity())
3587     // Dry Bulb Temperature
3588     .dryBulbT(row_data.getDryBulbTemp())
3589     // Wet Bulb Temperature
3590     .wetBulbT(row_data.getWetBulbTemp())
3591     // Dew Point
3592     .dewPt(row_data.getDewPoint())
3593     // RH
3594     .rh(row_data.getRh())
3595     // Reading Date
3596     .readDt(row_data.getReading_date())
3597     // Start Time
3598     .readTimeSt(row_data.getStartTime())
3599     // End Time
3600     .readTimeEnd(row_data.getEndTime())
3601     // Remark
3602     .inspRmrk(row_data.getRemark())
3603     // Cleaned By
3604     .cleanBy(form_input.getClean_used())
3605     // Rust Grade
3606     .rawRustGrade(form_input.getRaw_matl_rust_grade())
3607     // Dust Free Check Result
3608     .dust_free_check_result(form_input.getDust_free_check_result())
3609     // Visual Inspection
3610     .visInsp(form_input.getVisual_insp())
3611     // Cleared By
3612     .inspClrBy(authed_obj.getName())
3613     // Cleared On
3614     .inspClrDt(LocalDate.now())
3615     // Build !
3616     .build();
3617
3618 pms_ptool_dao.save(record);
3619
3620 /*
3621  * Deduct - Additional-Input-Given-Qty as the call is rejected in FNL_QTY
3622  * Table
3623  * for Tracking - Start ::: Rejection Handling - I
3624  */
3625 if (status.equals("reject")) {
3626     Pms_Fnl_Qty_PK pk_fnl = Pms_Fnl_Qty_PK.builder()
3627         // Vendor Code
3628         .vcode(form_input.getFrmGrpCtrlVcode())
3629         // PO
3630         .po(form_input.getFrmGrpCtrlPo())

```

```

3631         // DU_NUM
3632         .poSno(row_data.getDu())
3633         // Build!
3634         .build();
3635
3636         //////////////////////////////////////
3637
3638         Double existing_qty = 0.0;
3639
3640         Optional<Pms_Fnl_Qty> single_du_record_optional = pms_fnl_qty_dao.
            findById(pk_fnl);
3641
3642         if (single_du_record_optional.isPresent()) {
3643             existing_qty = single_du_record_optional.get().getAccQty();
3644         }
3645
3646         Double cumulative_qty = Math.abs(row_data.getQuantity() - existing_qty);
3647
3648         if (cumulative_qty == 0) {
3649             pms_fnl_qty_dao.deleteById(pk_fnl);
3650         } else {
3651             String currentYear = String.valueOf(LocalDate.now().getYear());
3652
3653             Pms_Fnl_Qty fnl = Pms_Fnl_Qty.builder()
3654                 // PK
3655                 .pms_fnl_pk(pk_fnl)
3656                 // Accepted Quantity
3657                 .accQty(cumulative_qty)
3658                 // Accepted Year
3659                 .pmsCallYr(currentYear)
3660                 // Build!
3661                 .build();
3662
3663             pms_fnl_qty_dao.save(fnl);
3664         }
3665
3666         //////////////////////////////////////
3667
3668     }
3669     /*
3670     * Deduct - Additional-Input-Given-Qty as the call is rejected in FNL_QTY
3671     Table
3672     * for Tracking - End
3673     */
3674 });
3675
3676 /**
3677 * Update Batch-Track Table Too to mark this call as completed only
3678 */
3679
3680 LocalDateTime currentTime = LocalDateTime.now();
3681 LocalDateTime nextCallTime = null;
3682 Integer callRefNum = form_input.getFrmGrpCtrlRef();
3683 String vcode = form_input.getFrmGrpCtrlVcode();
3684
3685 LocalDate today = LocalDate.now();
3686 int fyYear;
3687
3688 if (today.getMonthValue() < 4) { // Jan, Feb, Mar
3689     fyYear = today.getYear() - 1;

```

```

3693     } else { // Apr to Dec
3694         fyYear = today.getYear();
3695     }
3696
3697     String inspFyYear = String.valueOf(fyYear);
3698
3699     String callType = form_input.getFrm_grp_ctrl_call_type();
3700     String call_status = "";
3701
3702     if (status.equals("accept")) {
3703
3704         call_status = "ACC";
3705
3706         String paint_type = form_input.getFrm_grp_ctrl_paint_type();
3707
3708         PMS_Paint_Curing_Time paint_curing_data = paint_cur_dao.findById(paint_type).
            get();
3709
3710         nextCallTime = currentTime.plusHours(paint_curing_data.getCuring_time().
            longValue());
3711
3712     } else if (status.equals("reject")) {
3713
3714         call_status = "REJ";
3715
3716         nextCallTime = null;
3717
3718     } else if (status.equals("rework")) {
3719
3720         call_status = "REWORK";
3721
3722         nextCallTime = LocalDateTime.now();
3723     }
3724
3725     String actBy = authed_obj.getName();
3726
3727     // ::: Rejection Handling - II
3728     pms_btrack_dao.update_call_completion_for_a_call_type(currentTime, callRefNum,
        vcode, inspFyYear, callType,
3729         call_status, actBy);
3730
3731     /**
3732      * Reset PMS_ROOT table's CALL_STAT to 'ACC/REJ/REWORK' to indicate the call is
3733      * inspected
3734      *
3735      * ::: Rejection Handling - III
3736      */
3737     pms_root_dao.update_root_with_STAGE(call_status, vcode, form_input.
        getFrmGrpCtrlRef());
3738
3739     return true;
3740 }
3741
3742 /**
3743  * Record Blast-2.5 Call-Data
3744  */
3745 @Transactional(rollbackFor = Exception.class)
3746 public boolean record_blast_2_5_all(Pms_Blast_sa_2_5_Vo form_input, Authentication
    authed_obj, String status)
3747     throws Exception {
3748
3749     form_input.getRowFormGroup().forEach((row_data) -> {
3750
3751         Pms_Blast_2_5_PK pk = Pms_Blast_2_5_PK.builder()

```

```

3752         // Call Request Number
3753         .callRefNum(form_input.getFrmGrpCtrlRef())
3754         // Call's Vendor Code
3755         .vcode(form_input.getFrmGrpCtrlVcode())
3756         // Call's PO
3757         .poNum(form_input.getFrmGrpCtrlPo())
3758         // Call's DU - With Iteration
3759         .duNum(row_data.getDu())
3760         // Build !
3761         .build();
3762
3763     Pms_Blast_2_5 recrod = Pms_Blast_2_5.builder()
3764         /*****
3765         // Primary Key
3766         .blast_2_5_pk(pk)
3767         // DU Description
3768         .duDesc(row_data.getDuDescription())
3769         // DU Call Quantity
3770         .duCallQty(row_data.getQuantity())
3771         // Dry Bulb Temperature
3772         .dryBulbT(row_data.getDryBulbTemp())
3773         // Wet Bulb Temperature
3774         .wetBulbT(row_data.getWetBulbTemp())
3775         // Dew Point
3776         .dewPt(row_data.getDewPoint())
3777         // RH
3778         .rh(row_data.getRh())
3779         // Reading Date
3780         .readDt(row_data.getReading_date())
3781         // Start Time
3782         .readTimeSt(row_data.getStartTime())
3783         // End Time
3784         .readTimeEnd(row_data.getEndTime())
3785         // Remark
3786         .inspRmrk(row_data.getRemark())
3787         *****/
3788         // Blasting Medium
3789         .blast_medium(form_input.getBlast_medium())
3790         // Rust Grade
3791         .raw_matl_rust_grade(form_input.getRaw_matl_rust_grade())
3792         // Moist Content Test
3793         .moist_cont(form_input.getMoist_cont())
3794         // Surface Chloride
3795         .surf_cloride(form_input.getSurf_cloride())
3796         // Profile Gauge Reading
3797         .surf_prof_gauge_read(form_input.getSurf_prof_gauge_read())
3798         // Checked Date & Time
3799         .checked_dt_tm(form_input.getChecked_dt_tm())
3800         // Dust Free Check Result
3801         .dust_free_check(form_input.getDust_free_check())
3802         // Visual Inspection
3803         .visual_insp(form_input.getVisual_insp())
3804         /*****
3805         // Cleared By
3806         .inspClrBy(authed_obj.getName())
3807         // Cleared On
3808         .inspClrDt(LocalDate.now())
3809         *****/
3810         // Build !
3811         .build();
3812
3813     var saved_data = pms_blast_dao.save(recrod);
3814
3815     /*

```

```

3816         * Deduct - Additional-Input-Given-Qty as the call is rejected in FNL_QTY
3817         Table
3818         * for Tracking - Start ::: Rejection Handling - I
3819         */
3820     if (status.equals("reject")) {
3821         Pms_Fnl_Qty_PK pk_fnl = Pms_Fnl_Qty_PK.builder()
3822             // Vendor Code
3823             .vcode(form_input.getFrmGrpCtrlVcode())
3824             // PO
3825             .po(form_input.getFrmGrpCtrlPo())
3826             // DU_NUM
3827             .poSno(row_data.getDu())
3828             // Build!
3829             .build();
3830
3831         //////////////////////////////////////
3832
3833         Double existing_qty = 0.0;
3834
3835         Optional<Pms_Fnl_Qty> single_du_record_optional = pms_fnl_qty_dao.
3836             findById(pk_fnl);
3837
3838         if (single_du_record_optional.isPresent()) {
3839             existing_qty = single_du_record_optional.get().getAccQty();
3840         }
3841
3842         Double cumulative_qty = Math.abs(row_data.getQuantity() - existing_qty);
3843
3844         if (cumulative_qty == 0) {
3845             pms_fnl_qty_dao.deleteById(pk_fnl);
3846         } else {
3847             String currentYear = String.valueOf(LocalDate.now().getYear());
3848
3849             Pms_Fnl_Qty fnl = Pms_Fnl_Qty.builder()
3850                 // PK
3851                 .pms_fnl_pk(pk_fnl)
3852                 // Accepted Quantity
3853                 .accQty(cumulative_qty)
3854                 // Accepted Year
3855                 .pmsCallYr(currentYear)
3856                 // Build!
3857                 .build();
3858
3859             pms_fnl_qty_dao.save(fnl);
3860         }
3861
3862         //////////////////////////////////////
3863     }
3864     /*
3865     * Deduct - Additional-Input-Given-Qty as the call is rejected in FNL_QTY
3866     Table
3867     * for Tracking - End
3868     */
3869 }
3870
3871 /**
3872 * Update Batch-Track Table Too to mark this call as completed only

```

```

3877     */
3878     LocalDateTime currentTime = LocalDateTime.now();
3879     LocalDateTime nextCallTime = null;
3880     Integer callRefNum = form_input.getFrmGrpCtrlRef();
3881     String vcode = form_input.getFrmGrpCtrlVcode();
3882
3883     LocalDate today = LocalDate.now();
3884     int fyYear;
3885
3886     if (today.getMonthValue() < 4) { // Jan, Feb, Mar
3887         fyYear = today.getYear() - 1;
3888     } else { // Apr to Dec
3889         fyYear = today.getYear();
3890     }
3891
3892     String inspFyYear = String.valueOf(fyYear);
3893
3894     String callType = form_input.getFrm_grp_ctrl_call_type();
3895     String call_status = "";
3896
3897     if (status.equals("accept")) {
3898
3899         call_status = "ACC";
3900
3901         String paint_type = form_input.getFrm_grp_ctrl_paint_type();
3902
3903         PMS_Paint_Curing_Time paint_curing_data = paint_cur_dao.findById(paint_type).
3904             get();
3905
3906         nextCallTime = currentTime.plusHours(paint_curing_data.getCuring_time().
3907             longValue());
3908
3909     } else if (status.equals("reject")) {
3910
3911         call_status = "REJ";
3912
3913         nextCallTime = null;
3914
3915     } else if (status.equals("rework")) {
3916
3917         call_status = "REWORK";
3918
3919         nextCallTime = LocalDateTime.now();
3920     }
3921
3922     String actBy = authed_obj.getName();
3923
3924     // ::: Rejection Handling - II
3925     pms_btrack_dao.update_call_completion_for_a_call_type(currentTime, callRefNum,
3926         vcode, inspFyYear, callType,
3927         call_status, actBy);
3928
3929     /**
3930     * Reset PMS_ROOT table's CALL_STAT to 'ACC/REJ/REWORK' to indicate the call is
3931     * inspected
3932     *
3933     * ::: Rejection Handling - III
3934     */
3935     pms_root_dao.update_root_with_STAGE(call_status, vcode, form_input.
3936         getFrmGrpCtrlRef());
3937
3938     return true;
3939 }

```

```

3937  /**
3938  * Record Primary-Coat-I-II Call-Data
3939  */
3940  @Transactional(rollbackFor = Exception.class)
3941  public boolean record_inter_coat_I_II(Pms_Prime_I_II_Vo form_input, Authentication
    authed_data, String status)
3942      throws Exception {
3943
3944      form_input.getRowFormGroup().forEach((row_data) -> {
3945
3946          Pms_Prime_I_II_PK pk = Pms_Prime_I_II_PK.builder()
3947              // Call Request Number
3948              .callRefNum(form_input.getFrmGrpCtrlRef())
3949              // Call Type
3950              .callType(form_input.getFrm_grp_ctrl_call_type())
3951              // Call's Vendor Code
3952              .vcode(form_input.getFrmGrpCtrlVcode())
3953              // Call's PO
3954              .poNum(form_input.getFrmGrpCtrlPo())
3955              // Call's DU - With Iteration
3956              .duNum(row_data.getDu())
3957              // Build !
3958              .build();
3959
3960          /*****/
3961
3962          Pms_Prime_I_II recrod = Pms_Prime_I_II.builder()
3963              /*****/
3964              // Primary Key
3965              .pow_inter_coat_pk(pk)
3966              // DU Description
3967              .duDesc(row_data.getDuDescription())
3968              // DU Call Quantity
3969              .duCallQty(row_data.getQuantity())
3970              // Dry Bulb Temperature
3971              .dryBulbT(row_data.getDryBulbTemp())
3972              // Wet Bulb Temperature
3973              .wetBulbT(row_data.getWetBulbTemp())
3974              // Metal Surface Temperature
3975              .matelSurfT(row_data.getExtra())
3976              // Dew Point
3977              .dewPt(row_data.getDewPoint())
3978              // RH
3979              .rh(row_data.getRh())
3980              // Reading Date
3981              .readDt(row_data.getReading_date())
3982              // Start Time
3983              .readTimeSt(row_data.getStartTime())
3984              // End Time
3985              .readTimeEnd(row_data.getEndTime())
3986              // Remark
3987              .inspRmrk(row_data.getRemark())
3988              /*****/
3989              // Paint Specification
3990              .paint_spec(form_input.getPaint_spec())
3991              // paint Shade
3992              .paint_shade(form_input.getPaint_shade())
3993              // Paint Manufacturer
3994              .paint_manuf(form_input.getPaint_manuf())
3995              // Supplier/ Trader
3996              .supplier_trader(form_input.getSupplier_trader())
3997              // Paint Manufacturing Date
3998              .manuf_dt(form_input.getManuf_dt())
3999              // Manufacturing Batch

```



```

4000         .manuf_batch(form_input.getManuf_batch())
4001         // Paint TC
4002         .manuf_tc(form_input.getManuf_tc())
4003         // Paint Expire Date
4004         .paint_exp_dt(form_input.getPaint_exp_dt())
4005         // Mix-Ratio
4006         .mix_ratio(form_input.getMix_ratio())
4007         // Number of Coat
4008         .num_coat(form_input.getNum_coat())
4009         // Duration of Mix
4010         .time_of_mix(form_input.getTime_of_mix())
4011         // Wet Film Thickness
4012         .wet_film_thick(form_input.getWet_film_thick())
4013         // Spray Gun
4014         .spary_gun(form_input.getSpary_gun())
4015         // Hard Dry Time Required
4016         .hard_dry_time_req(form_input.getHard_dry_time_req())
4017         // DFT Required
4018         .dft_req(form_input.getDft_req())
4019         // DTF Actual Measurement
4020         .dft_act(form_input.getDft_act())
4021         // Visual Inspection
4022         .visual_insp(form_input.getVisual_insp())
4023         // Cross Cut Test upto 50u
4024         .cross_cut_dft_50(form_input.getCross_cut_dft_50())
4025         // Cross Cut Test 50 - 125u
4026         .cross_cut_dft_50_125(form_input.getCross_cut_dft_50_125())
4027         // Cross Cut Test above 125u
4028         .cross_cut_dft_125(form_input.getCross_cut_dft_125())
4029         /*****
4030         // Cleared By
4031         .inspClrBy(authed_data.getName())
4032         // Cleared On
4033         .inspClrDt(LocalDate.now())
4034         *****/
4035         // Build !
4036         .build();
4037
4038     var saved_data = pms_inter_coat_dao.save(recrod);
4039
4040     /*
4041     * Deduct - Additional-Input-Given-Qty as the call is rejected in FNL_QTY
4042     Table
4043     * for Tracking - Start ::: Rejection Handling - I
4044     */
4045     if (status.equals("reject")) {
4046         Pms_Fnl_Qty_PK pk_fnl = Pms_Fnl_Qty_PK.builder()
4047             // Vendor Code
4048             .vcode(form_input.getFrmGrpCtrlVcode())
4049             // PO
4050             .po(form_input.getFrmGrpCtrlPo())
4051             // DU_NUM
4052             .poSno(row_data.getDu())
4053             // Build!
4054             .build();
4055
4056         //////////////////////////////////////
4057
4058         Double existing_qty = 0.0;
4059
4060         Optional<Pms_Fnl_Qty> single_du_record_optional = pms_fnl_qty_dao.
4061             findById(pk_fnl);

```

```

4062         if (single_du_record_optional.isPresent()) {
4063             existing_qty = single_du_record_optional.get().getAccQty();
4064         }
4065         Double cumulative_qty = Math.abs(row_data.getQuantity() - existing_qty);
4066
4067         if (cumulative_qty == 0) {
4068             pms_fnl_qty_dao.deleteById(pk_fnl);
4069         } else {
4070             String currentYear = String.valueOf(LocalDate.now().getYear());
4071
4072             Pms_Fnl_Qty fnl = Pms_Fnl_Qty.builder()
4073                 // PK
4074                 .pms_fnl_pk(pk_fnl)
4075                 // Accepted Quantity
4076                 .accQty(cumulative_qty)
4077                 // Accepted Year
4078                 .pmsCallYr(currentYear)
4079                 // Build!
4080                 .build();
4081
4082             pms_fnl_qty_dao.save(fnl);
4083         }
4084
4085         //////////////////////////////////////////
4086     }
4087     /*
4088     * Deduct - Additional-Input-Given-Qty as the call is rejected in FNL_QTY
4089     Table
4090     * for Tracking - End
4091     */
4092
4093 });
4094
4095 /**
4096  * Update Batch-Track Table Too to mark this call as completed only
4097  */
4098
4099 LocalDateTime currentTime = LocalDateTime.now();
4100 LocalDateTime nextCallTime = null;
4101 Integer callRefNum = form_input.getFrmGrpCtrlRef();
4102 String vcode = form_input.getFrmGrpCtrlVcode();
4103
4104 LocalDate today = LocalDate.now();
4105 int fyYear;
4106
4107 if (today.getMonthValue() < 4) { // Jan, Feb, Mar
4108     fyYear = today.getYear() - 1;
4109 } else { // Apr to Dec
4110     fyYear = today.getYear();
4111 }
4112
4113 String inspFyYear = String.valueOf(fyYear);
4114
4115 String callType = form_input.getFrm_grp_ctrl_call_type();
4116 String call_status = "";
4117
4118 if (status.equals("accept")) {
4119

```

```

4125         call_status = "ACC";
4126
4127         String paint_type = form_input.getFrm_grp_ctrl_paint_type();
4128
4129         PMS_Paint_Curing_Time paint_curing_data = paint_cur_dao.findById(paint_type).
get();
4130
4131         nextCallTime = currentTime.plusHours(paint_curing_data.getCuring_time().
longValue());
4132
4133         System.out.println("Curing hours: " + paint_curing_data.getCuring_time());
4134         System.out.println("Current time: " + currentTime);
4135         System.out.println("Next call time: " + nextCallTime);
4136
4137     } else if (status.equals("reject")) {
4138
4139         call_status = "REJ";
4140
4141         nextCallTime = null;
4142
4143     } else if (status.equals("rework")) {
4144
4145         call_status = "REWORK";
4146
4147         nextCallTime = LocalDateTime.now();
4148     }
4149
4150     String actBy = authed_data.getName();
4151
4152     // ::: Rejection Handling - II
4153     pms_btrack_dao.update_call_completion_for_a_call_type(currentTime, callRefNum,
vcode, inspFyYear, callType,
call_status, actBy);
4154
4155
4156     /**
4157     * Reset PMS_ROOT table's CALL_STAT to 'ACC/REJ/REWORK' to indicate the call is
4158     * inspected
4159     *
4160     * ::: Rejection Handling - III
4161     */
4162     pms_root_dao.update_root_with_STAGE(call_status, vcode, form_input.
getFrmGrpCtrlRef());
4163
4164     return true;
4165 }
4166
4167 /**
4168 * Get PMS-FNL-QTY Data for the Vendor-Po
4169 */
4170 public List<Pms_Fnl_Qty_Dto> get_pms_fnl_qty_for_a_po(String vcode) {
4171
4172     return pms_fnl_qty_dao.get_pms_fnl_qty_for_a_po(vcode);
4173 }
4174
4175
4176 /**
4177 * Record Batch-Map-To-A-Call-Type
4178 *
4179 */
4180 @Transactional(rollbackFor = Exception.class)
4181 public PMS_Batch_Track record_batch_to_call_type(String batch, String call_type,
String paint_type,
String call_dt_tm, MultipartFile doc_p, Authentication authed_obj) throws
Exception {
4182

```

```

4183
4184 DateTimeFormatter formatter = DateTimeFormatter.ofPattern("yyyy-MM-dd'T'HH:mm");
4185 LocalDateTime call_req_dt_time = LocalDateTime.parse(call_dt_tm, formatter);
4186
4187 String fileName = String.format("%S_%S_%S_%S", authed_obj.getName(), batch,
call_type,
String.valueOf(LocalDateTime.now().getYear()));
4188
4189
4190 fileName = fileName.replaceAll("[\\s\\-\\.]", "_");
4191
4192 fileName = fileName + ".pdf";
4193
4194 if (doc_p == null) {
4195     fileName = "";
4196 }
4197
4198 LocalDate today = LocalDate.now();
4199 int fyYear;
4200
4201 if (today.getMonthValue() < 4) { // Jan, Feb, Mar
4202     fyYear = today.getYear() - 1;
4203 } else { // Apr to Dec
4204     fyYear = today.getYear();
4205 }
4206
4207 String inspFyYear = String.valueOf(fyYear);
4208
4209 /**
4210  * Primary key
4211  */
4212 PMS_Batch_Track_PK pk = PMS_Batch_Track_PK.builder()
4213     // Vendor Code
4214     .vcode(authed_obj.getName())
4215     // Batch
4216     .callRefNum(Integer.parseInt(batch))
4217     // Call-Request Date-Time
4218     .callReqDtTm(call_req_dt_time)
4219     // Call-Type
4220     .callType(call_type)
4221     // Paint-Type
4222     .inspFyYear(inspFyYear)
4223     // Build!
4224     .build();
4225
4226 /**
4227  * Object
4228  */
4229 PMS_Batch_Track row = PMS_Batch_Track.builder()
4230     // Primary Key
4231     .btrack_pk(pk)
4232     // Paint-Type
4233     .paintType(paint_type)
4234     // Document name
4235     .paint_tc_doc(fileName)
4236     // Build!
4237     .build();
4238
4239 PMS_Batch_Track result = pms_btrack_dao.save(row);
4240
4241 /**
4242  * Update PMS_ROOT for Current Call_Type & Paint_Type
4243  */
4244 pms_root_dao.update_root_with_CURRENT_Call_Type_Map("MAPPED_BATCH", call_type,
paint_type, fileName,

```

```

4245         call_req_dt_time, authed_obj.getName(), Integer.parseInt(batch));
4246
4247     if (result != null && doc_p != null) {
4248
4249         // Initialize here, AFTER @Value injection
4250         Path uploadPath = Paths.get(UPLOAD_DIR + "PMS/TC_Docs/");
4251
4252         if (!Files.exists(uploadPath)) {
4253             try {
4254                 Files.createDirectories(uploadPath);
4255             } catch (IOException e) {
4256                 e.printStackTrace();
4257             }
4258             System.out.println("Created upload directory: " + uploadPath.
                toAbsolutePath());
4259         }
4260
4261         Path filePath = uploadPath.resolve(fileName);
4262
4263         try {
4264             Files.copy(doc_p.getInputStream(), filePath, StandardCopyOption.
                REPLACE_EXISTING);
4265         } catch (IOException e) {
4266             e.printStackTrace();
4267         }
4268     }
4269
4270     return result;
4271 }
4272
4273 /**
4274  * PMS Call Report for a Vendor
4275  */
4276 public List<Pms_Call_Report_Dto> pms_call_report_for_a_vendor(String vcode) {
4277
4278     return pms_root_dao.pms_call_report_for_a_vendor(vcode);
4279 }
4280
4281 /**
4282  * PMS Call Report for ALL Vendor
4283  */
4284 public List<Pms_Call_Report_Dto> pms_call_report_ALL_vendor() {
4285
4286     return pms_root_dao.pms_call_report_all_vendor();
4287 }
4288
4289 /**
4290  * Completed/WIP Call-Type for a given given batch (Call History Tracking!!!)
4291  */
4292 public List<PMS_Batch_Track_Dto> get_call_type_history_for_a_vendor(String vcode) {
4293
4294     return pms_btrack_dao.get_call_type_history_for_a_vendor(vcode);
4295 }
4296
4297 /**
4298  * Get Completed call-list for printing & signing
4299  */
4300 public List<PMS_Batch_Track_Dto> get_completed_call_list() {
4301
4302     return pms_btrack_dao.get_completed_call_list();
4303 }
4304
4305 /**
4306  * For Power-Tool-Cleaning Accepted Data

```

```

4307     */
4308     public List<Pms_Tpi_Dynamic_Row_Dto>
4309     get_call_type_spcific_data_powr_tool_cleaning_serv(String vcode,
4310         String callNum, String inspYr) {
4311
4312         return pms_ptool_dao.get_dynamic_row_data(vcode, Integer.parseInt(callNum));
4313     }
4314
4315     /**
4316     * For Blasting Accepted Data
4317     */
4318     public List<Pms_Tpi_Dynamic_Row_Dto> get_call_type_spcific_data_blasting_serv(String
4319     vcode, String callNum,
4320         String inspYr) {
4321
4322         return pms_blast_dao.get_dynamic_row_data(vcode, Integer.parseInt(callNum));
4323     }
4324
4325     /**
4326     * For Primary & Other call-type Accepted Data
4327     */
4328     public List<Pms_Tpi_Dynamic_Row_Dto> get_call_type_spcific_data_prime_others_serv(
4329     String vcode, String callNum,
4330         String inspYr, String callType) {
4331
4332         return pms_inter_coat_dao.get_dynamic_row_data(vcode, Integer.parseInt(callNum),
4333             callType);
4334     }
4335
4336     /**
4337     * Get Image Name & location for Each DU
4338     */
4339     public List<Pms_Du_Img_GLloc_Dto> get_image_name_location_data(String vcode, int
4340     callNum, String callType) {
4341
4342         var result = pms_geo.get_image_name_location_data(vcode, callNum, callType);
4343
4344         return result;
4345     }
4346
4347     package com.example.demo.model.dto.insp;
4348
4349     import java.math.BigDecimal;
4350     import java.time.LocalDate;
4351     import java.time.LocalDateTime;
4352     import java.time.LocalTime;
4353
4354     import com.example.demo.model.bo.insp.Pms_Blast_2_5;
4355     import com.example.demo.model.bo.insp.Pms_Pow_Tool;
4356     import com.example.demo.model.bo.insp.Pms_Prime_I_II;
4357     import com.example.demo.model.bo.outs.Po_Details;
4358
4359     import jakarta.persistence.Column;
4360     import lombok.AllArgsConstructor;
4361     import lombok.Builder;
4362     import lombok.Data;
4363     import lombok.NoArgsConstructor;
4364
4365     @NoArgsConstructor

```

```

4366 @AllArgsConstructor
4367 @Data
4368 @Builder
4369 public class Pms_Tpi_Dynamic_Row_Dto {
4370
4371     /**
4372      * Common for All Call-Type
4373      */
4374     private Integer duNum;
4375     private String duDesc;
4376     private double duCallQty;
4377     private BigDecimal dryBulbT;
4378     private BigDecimal wetBulbT;
4379     private BigDecimal extra = BigDecimal.ZERO;
4380     private BigDecimal dewPt;
4381     private BigDecimal rh;
4382     private LocalDate readDt;
4383     private LocalTime readTimeSt;
4384     private LocalTime readTimeEnd;
4385     private String inspRmrk;
4386     private String du_code = "";
4387
4388     /**
4389      *
4390      * Specific for 'Power Tool Cleaning'
4391      */
4392     private String cleaned_by = "";
4393     private String raw_mat_rust_grade_ptc = "";
4394     private String dust_free_check_result = "";
4395     private String visual_insp = "";
4396
4397     /**
4398      *
4399      * Specific for 'Blasting SA 2.5'
4400      */
4401     private String blast_by = "";
4402     private String raw_mat_rust_grade_blast = "";
4403     private String moist_cont;
4404     private String surf_chloride;
4405     private Double surf_prof_gauge_read;
4406     private LocalDateTime checked_dt_tm;
4407     private String dust_free_check;
4408     private String visual_insp_blast;
4409
4410     /**
4411      *
4412      * Specific for 'Prime' & Others
4413      */
4414     private String paint_spec = "";
4415     private String paint_shade = "";
4416     private String paint_manuf = "";
4417     private String paint_supp_trade = "";
4418     private LocalDate paint_manu_dt;
4419     private String paint_manu_batch = "";
4420     private String paint_tc_num = "";
4421     private LocalDate paint_exp_dt;
4422     private String mix_ratio = "";
4423     private String no_of_coats = "";
4424     private String mix_duration = "";
4425     private String wet_film_thick = "";
4426     private String spray_gun = "";
4427     private String hard_dry_temp_req = "";
4428     private String dft_req = "";
4429     private String dft_act;

```

```

4430 private String visual_insp_prime;
4431 private String cross_cut_dft_50;
4432 private String cross_cut_dft_50_125;
4433 private String cross_cut_dft_125;
4434
4435 /*****
4436
4437 /**
4438  *
4439  * Specific for 'Power Tool Cleaning'
4440  */
4441 public Pms_Tpi_Dynamic_Row_Dto(Pms_Pow_Tool p) {
4442
4443     this.duNum = p.getPow_tool_pk().getDuNum();
4444     this.duDesc = p.getDuDesc();
4445     this.duCallQty = p.getDuCallQty();
4446     this.dryBulbT = p.getDryBulbT();
4447     this.wetBulbT = p.getWetBulbT();
4448     this.dewPt = p.getDewPt();
4449     this.rh = p.getRh();
4450     this.readDt = p.getReadDt();
4451     this.readTimeSt = p.getReadTimeSt();
4452     this.readTimeEnd = p.getReadTimeEnd();
4453     this.inspRmrk = p.getInspRmrk();
4454
4455     this.cleaned_by = p.getCleanBy();
4456     this.raw_mat_rust_grade_ptc = p.getRawRustGrade();
4457     this.dust_free_check_result = p.getDust_free_check_result();
4458     this.visual_insp = p.getVisInsp();
4459
4460 }
4461
4462 /**
4463  *
4464  * Specific for 'Blasting SA 2.5'
4465  */
4466 public Pms_Tpi_Dynamic_Row_Dto(Pms_Blast_2_5 p) {
4467
4468     this.duNum = p.getBlast_2_5_pk().getDuNum();
4469     this.duDesc = p.getDuDesc();
4470     this.duCallQty = p.getDuCallQty();
4471     this.dryBulbT = p.getDryBulbT();
4472     this.wetBulbT = p.getWetBulbT();
4473     this.dewPt = p.getDewPt();
4474     this.rh = p.getRh();
4475     this.readDt = p.getReadDt();
4476     this.readTimeSt = p.getReadTimeSt();
4477     this.readTimeEnd = p.getReadTimeEnd();
4478     this.inspRmrk = p.getInspRmrk();
4479
4480     this.blast_by = p.getBlast_medium();
4481     this.raw_mat_rust_grade_blast = p.getRaw_matl_rust_grade();
4482     this.moist_cont = p.getMoist_cont();
4483     this.surf_cloride = p.getSurf_cloride();
4484     this.surf_prof_gauge_read = p.getSurf_prof_gauge_read();
4485     this.checked_dt_tm = p.getChecked_dt_tm();
4486     this.dust_free_check = p.getDust_free_check();
4487     this.visual_insp_blast = p.getVisual_insp();
4488
4489 }
4490
4491 /**
4492  *
4493  * Specific for 'Prime' & Others

```



```

4494 */
4495 public Pms_Tpi_Dynamic_Row_Dto(Pms_Prime_I_II p) {
4496
4497     this.duNum = p.getPow_inter_coat_pk().getDuNum();
4498     this.duDesc = p.getDuDesc();
4499     this.duCallQty = p.getDuCallQty();
4500     this.dryBulbT = p.getDryBulbT();
4501     this.wetBulbT = p.getWetBulbT();
4502     this.extra = p.getMatelSurfT();
4503     this.dewPt = p.getDewPt();
4504     this.rh = p.getRh();
4505     this.readDt = p.getReadDt();
4506     this.readTimeSt = p.getReadTimeSt();
4507     this.readTimeEnd = p.getReadTimeEnd();
4508     this.inspRmrk = p.getInspRmrk();
4509
4510     this.paint_spec = p.getPaint_spec();
4511     this.paint_shade = p.getPaint_shade();
4512     this.paint_manuf = p.getPaint_manuf();
4513     this.paint_supp_trade = p.getSupplier_trader();
4514     this.paint_manu_dt = p.getManuf_dt();
4515     this.paint_manu_batch = p.getManuf_batch();
4516     this.paint_tc_num = p.getManuf_tc();
4517     this.paint_exp_dt = p.getPaint_exp_dt();
4518     this.mix_ratio = p.getMix_ratio();
4519     this.no_of_coats = p.getNum_coat();
4520     this.mix_duration = p.getTime_of_mix();
4521     this.wet_film_thick = p.getWet_film_thick();
4522     this.spray_gun = p.getSpary_gun();
4523     this.hard_dry_temp_req = p.getHard_dry_time_req();
4524     this.dft_req = p.getDft_req();
4525     this.dft_act = p.getDft_act();
4526     this.visual_insp_prime = p.getVisual_insp();
4527     this.cross_cut_dft_50 = p.getCross_cut_dft_50();
4528     this.cross_cut_dft_50_125 = p.getCross_cut_dft_50_125();
4529     this.cross_cut_dft_125 = p.getCross_cut_dft_125();
4530 }
4531
4532 /*****
4533
4534 /**
4535  *
4536  * Specific for 'Power Tool Cleaning'
4537  */
4538 public Pms_Tpi_Dynamic_Row_Dto(Pms_Pow_Tool p, Po_Details q) {
4539
4540     this.duNum = p.getPow_tool_pk().getDuNum();
4541     this.duDesc = p.getDuDesc();
4542     this.duCallQty = p.getDuCallQty();
4543     this.dryBulbT = p.getDryBulbT();
4544     this.wetBulbT = p.getWetBulbT();
4545     this.dewPt = p.getDewPt();
4546     this.rh = p.getRh();
4547     this.readDt = p.getReadDt();
4548     this.readTimeSt = p.getReadTimeSt();
4549     this.readTimeEnd = p.getReadTimeEnd();
4550     this.inspRmrk = p.getInspRmrk();
4551     this.du_code = q.getZzDuCode();
4552
4553     this.cleaned_by = p.getCleanBy();
4554     this.raw_mat_rust_grade_ptc = p.getRawRustGrade();
4555     this.dust_free_check_result = p.getDust_free_check_result();
4556     this.visual_insp = p.getVisInsp();
4557

```

```

4558 }
4559
4560 /**
4561  *
4562  * Specific for 'Blasting SA 2.5'
4563  */
4564 public Pms_Tpi_Dynamic_Row_Dto(Pms_Blast_2_5 p, Po_Details q) {
4565
4566     this.duNum = p.getBlast_2_5_pk().getDuNum();
4567     this.duDesc = p.getDuDesc();
4568     this.duCallQty = p.getDuCallQty();
4569     this.dryBulbT = p.getDryBulbT();
4570     this.wetBulbT = p.getWetBulbT();
4571     this.dewPt = p.getDewPt();
4572     this.rh = p.getRh();
4573     this.readDt = p.getReadDt();
4574     this.readTimeSt = p.getReadTimeSt();
4575     this.readTimeEnd = p.getReadTimeEnd();
4576     this.inspRmrk = p.getInspRmrk();
4577     this.du_code = q.getZzDuCode();
4578
4579     this.blast_by = p.getBlast_medium();
4580     this.raw_mat_rust_grade_blast = p.getRaw_matl_rust_grade();
4581     this.moist_cont = p.getMoist_cont();
4582     this.surf_cloride = p.getSurf_cloride();
4583     this.surf_prof_gauge_read = p.getSurf_prof_gauge_read();
4584     this.checked_dt_tm = p.getChecked_dt_tm();
4585     this.dust_free_check = p.getDust_free_check();
4586     this.visual_insp_blast = p.getVisual_insp();
4587
4588 }
4589
4590 /**
4591  *
4592  * Specific for 'Prime' & Others
4593  */
4594 public Pms_Tpi_Dynamic_Row_Dto(Pms_Prime_I_II p, Po_Details q) {
4595
4596     this.duNum = p.getPow_inter_coat_pk().getDuNum();
4597     this.duDesc = p.getDuDesc();
4598     this.duCallQty = p.getDuCallQty();
4599     this.dryBulbT = p.getDryBulbT();
4600     this.wetBulbT = p.getWetBulbT();
4601     this.extra = p.getMatelSurfT();
4602     this.dewPt = p.getDewPt();
4603     this.rh = p.getRh();
4604     this.readDt = p.getReadDt();
4605     this.readTimeSt = p.getReadTimeSt();
4606     this.readTimeEnd = p.getReadTimeEnd();
4607     this.inspRmrk = p.getInspRmrk();
4608     this.du_code = q.getZzDuCode();
4609
4610     this.paint_spec = p.getPaint_spec();
4611     this.paint_shade = p.getPaint_shade();
4612     this.paint_manuf = p.getPaint_manuf();
4613     this.paint_supp_trade = p.getSupplier_trader();
4614     this.paint_manu_dt = p.getManuf_dt();
4615     this.paint_manu_batch = p.getManuf_batch();
4616     this.paint_tc_num = p.getManuf_tc();
4617     this.paint_exp_dt = p.getPaint_exp_dt();
4618     this.mix_ratio = p.getMix_ratio();
4619     this.no_of_coats = p.getNum_coat();
4620     this.mix_duration = p.getTime_of_mix();
4621     this.wet_film_thick = p.getWet_film_thick();

```

```
4622         this.spray_gun = p.getSpary_gun();
4623         this.hard_dry_temp_req = p.getHard_dry_time_req();
4624         this.dft_req = p.getDft_req();
4625         this.dft_act = p.getDft_act();
4626         this.visual_insp_prime = p.getVisual_insp();
4627         this.cross_cut_dft_50 = p.getCross_cut_dft_50();
4628         this.cross_cut_dft_50_125 = p.getCross_cut_dft_50_125();
4629         this.cross_cut_dft_125 = p.getCross_cut_dft_125();
4630     }
4631 }
4632
```